

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO CUỐI KÌ
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

Lời cảm ơn

Trước hết, em xin gửi lời cảm ơn chân thành đến thầy Kim Ngọc Bách đã hướng dẫn, định hướng và hỗ trợ em trong quá trình thực hiện môn Thực tập cơ sở. Những góp ý và sự hướng dẫn của thầy đã giúp em hiểu rõ hơn về cách lựa chọn đề tài, xây dựng kế hoạch thực hiện, triển khai hệ thống và hoàn thiện báo cáo theo đúng yêu cầu.

Trong quá trình thực hiện đề tài “Xây dựng hệ thống web phát hiện người không đội mũ bảo hiểm bằng mô hình YOLOv8”, em đã có cơ hội vận dụng các kiến thức đã học vào một bài toán thực tế, đồng thời tìm hiểu thêm nhiều công nghệ mới như YOLOv8, FastAPI, React, MongoDB, PostgreSQL, WebSocket và các kỹ thuật xử lý ảnh, video thời gian thực. Đây là trải nghiệm quan trọng giúp em nâng cao khả năng tự học, tư duy phân tích hệ thống và kỹ năng phát triển phần mềm.

Do thời gian thực hiện và kinh nghiệm thực tế của bản thân còn hạn chế, báo cáo và hệ thống không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy để có thể tiếp tục hoàn thiện đề tài tốt hơn trong tương lai.

Em xin chân thành cảm ơn!

Mục lục

1. Giới thiệu chung.....	5
1.1. Tên đề tài.....	5
1.2. Bối cảnh và lý do chọn đề tài.....	5
1.3. Mục tiêu của đề tài.....	6
1.4. Phạm vi thực hiện.....	7
1.4.1. Phạm vi về mô hình AI.....	7
1.4.2. Phạm vi về Backend.....	8
1.4.3. Phạm vi về Frontend.....	8
1.4.4. Phạm vi về lưu trữ dữ liệu.....	9
1.5. Quá trình thực hiện đề tài.....	9
1.6. Ý nghĩa của đề tài.....	10
2. Cơ sở lý thuyết và công nghệ sử dụng.....	10
2.1. Thị giác máy tính.....	10
2.2. Bài toán Object Detection.....	11
2.3. Học sâu và mạng nơ-ron tích chập.....	12
2.4. Mô hình YOLO.....	12
2.5. Dataset và định dạng dữ liệu YOLO.....	13
2.6. Huấn luyện mô hình và Transfer Learning.....	14
2.7. Các chỉ số đánh giá mô hình.....	14
2.7.1. Precision.....	14
2.7.2. Recall.....	15
2.7.3. IoU.....	15
2.7.4. mAP.....	15
2.8. Tracking và ByteTrack trong video realtime.....	16
2.9. Xử lý ảnh và video bằng OpenCV.....	16
2.10. Streaming video thời gian thực.....	17
2.11. Backend FastAPI.....	17
2.12. Frontend React/Vite.....	18
2.13. Cơ sở dữ liệu PostgreSQL và SQLAlchemy.....	18
2.14. Alembic và quản lý migration database.....	19
2.15. MongoDB và dữ liệu AI.....	19
2.16. Cloudinary và lưu trữ ảnh bằng chứng.....	19
2.17. Xác thực, phân quyền và bảo mật.....	20
2.18. WebSocket và dữ liệu realtime.....	21
2.19. Gửi Email và SMTP.....	21
2.20. Các công nghệ chính sử dụng trong hệ thống.....	22
2.21. Nhận xét về lựa chọn công nghệ.....	23
3. Phân tích và thiết kế hệ thống.....	23

3.1. Tổng quan hệ thống.....	23
3.2. Phân tích yêu cầu hệ thống.....	24
3.2.1. Yêu cầu chức năng.....	24
3.2.1.1. Nhóm chức năng xác thực và phân quyền.....	24
3.2.1.2. Nhóm chức năng nhận diện ảnh.....	24
3.2.1.3. Nhóm chức năng giám sát livestream.....	25
3.2.1.4. Nhóm chức năng quản lý vi phạm.....	25
3.2.1.4. Nhóm chức năng thống kê và báo cáo.....	26
3.2.1.6. Nhóm chức năng quản lý camera.....	26
3.2.1.7. Nhóm chức năng Audit Log.....	26
3.2.2. Yêu cầu phi chức năng.....	27
3.2.2.1. Hiệu năng realtime.....	27
3.2.2.2. Tính ổn định.....	27
3.2.2.3. Tính bảo mật.....	27
3.2.2.4. Tính mở rộng.....	28
3.2.2.5. Tính dễ bảo trì.....	28
3.3. Nhóm người sử dụng hệ thống.....	28
3.3.1. Admin.....	28
3.3.2. Guard.....	28
3.4. Thiết kế kiến trúc tổng thể.....	29
3.5. Thiết kế kiến trúc Backend.....	30
3.5.1. Cấu trúc Backend.....	30
3.5.4. Thiết kế xử lý AI Model.....	31
3.6. Thiết kế Frontend.....	31
3.6.1. Cấu trúc Frontend.....	31
3.6.2. Thiết kế các trang chính.....	32
3.6.2.1. Login.....	32
3.6.2.2. Verify Email.....	32
3.6.2.3. Reset Password.....	32
3.6.2.4. Live Monitoring.....	32
3.6.2.5. Violation History.....	32
3.6.2.6. Analytics.....	32
3.6.2.7. User Management.....	33
3.6.2.8. Camera Management.....	33
3.6.2.9. Audit Logs.....	33
3.6.3. Thiết kế Route Protection.....	33
3.6.4. Thiết kế giao tiếp API.....	33
3.7. Thiết kế dữ liệu.....	34
3.7.1. Dữ liệu PostgreSQL.....	34
3.7.1.1. User.....	34

3.7.1.2. Refresh Token.....	34
3.7.1.3. Camera.....	35
3.7.1.4. Audit Log.....	35
3.7.1.5. Security Alert.....	36
3.7.2. Dữ liệu MongoDB.....	36
3.7.2.1. Collection violations.....	36
3.7.2.2. Collection traffic_stats.....	37
3.7.3. Dữ liệu Cloudinary.....	37
3.8. Thiết kế các luồng xử lý chính.....	38
3.8.1. Luồng đăng nhập.....	38
3.8.2. Luồng refresh token.....	38
3.8.3. Luồng xử lý ảnh upload.....	38
3.8.4. Luồng livestream và phát hiện vi phạm.....	39
3.8.5. Luồng kiểm duyệt vi phạm.....	39
3.8.6. Luồng quản lý camera.....	39
3.8.7. Luồng thống kê và báo cáo.....	40
3.9. Thiết kế AI Pipeline trong livestream.....	40
3.9.2. Cooldown và Dedup.....	41
3.10. Thiết kế bảo mật hệ thống.....	41
3.10.2. Refresh Token Rotation.....	42
3.11. Thiết kế triển khai và cấu hình.....	42
3.12. Nhận xét về thiết kế hệ thống.....	43
4. Kết quả cài đặt và thử nghiệm.....	43
4.1. Môi trường cài đặt và triển khai thử nghiệm.....	43
4.2. Kết quả huấn luyện và đánh giá mô hình AI.....	44
4.2.1. Kết quả huấn luyện YOLOv8n.....	44
4.2.2. Kết quả huấn luyện YOLOv8s 100 epochs.....	45
4.2.3. Kết quả huấn luyện YOLOv8s 200 epochs.....	45
4.2.4. Nhận xét về kết quả mô hình.....	46
4.3. Kết quả cài đặt hệ thống web.....	47
5. Kết luận.....	51
5.1. Nhận xét chung về hệ thống.....	51
5.2. Kết quả đạt được.....	52
5.3. Hạn chế của hệ thống.....	53
5.4. Hướng phát triển trong tương lai.....	54
5.5. Kết luận.....	55

1. Giới thiệu chung

1.1. Tên đề tài

Tên đề tài thực hiện trong môn Thực tập cơ sở là:

Xây dựng hệ thống web phát hiện người không đội mũ bảo hiểm bằng mô hình YOLOv8

Đề tài tập trung xây dựng một hệ thống ứng dụng trí tuệ nhân tạo vào bài toán giám sát an toàn giao thông, cụ thể là phát hiện người tham gia giao thông không đội mũ bảo hiểm thông qua ảnh, video hoặc camera thời gian thực. Hệ thống không chỉ dừng lại ở việc huấn luyện mô hình AI, mà còn được phát triển thành một ứng dụng web hoàn chỉnh, có khả năng giám sát livestream, lưu trữ bằng chứng vi phạm, kiểm duyệt dữ liệu, thống kê báo cáo, quản lý người dùng, quản lý camera và ghi nhật ký thao tác hệ thống.

1.2. Bối cảnh và lý do chọn đề tài

Trong thực tế, xe máy là phương tiện giao thông phổ biến tại Việt Nam. Việc đội mũ bảo hiểm khi tham gia giao thông là yêu cầu quan trọng nhằm bảo vệ an toàn cho người điều khiển và người ngồi sau xe máy. Tuy nhiên, tại nhiều khu vực, việc giám sát hành vi không đội mũ bảo hiểm vẫn còn phụ thuộc nhiều vào con người, camera quan sát thủ công hoặc các biện pháp kiểm tra trực tiếp.

Cách giám sát truyền thống tồn tại một số hạn chế:

- Cần có người theo dõi liên tục trong thời gian dài.
- Dễ bỏ sót vi phạm do mệt mỏi hoặc số lượng camera lớn.
- Khó tổng hợp dữ liệu vi phạm theo ngày, tháng hoặc khoảng thời gian.
- Khó lưu trữ bằng chứng một cách có hệ thống.
- Khó đánh giá xu hướng vi phạm hoặc hiệu quả giám sát.
- Khó mở rộng khi cần quản lý nhiều camera tại nhiều vị trí khác nhau

Trong khi đó, sự phát triển của thị giác máy tính, học sâu và các mô hình phát hiện vật thể thời gian thực như YOLO đã mở ra khả năng tự động hóa một phần quá trình giám sát. Thay vì con người phải quan sát toàn bộ video, hệ thống có thể tự động nhận diện đối tượng, phát hiện trường hợp không đội mũ bảo hiểm, lưu lại bằng chứng và hỗ trợ người dùng kiểm duyệt lại kết quả.

Từ nhu cầu thực tế đó, em lựa chọn đề tài này nhằm kết hợp giữa mô hình AI và hệ thống web để xây dựng một sản phẩm có tính ứng dụng rõ ràng. Quá trình thực hiện

đề tài giúp em không chỉ tìm hiểu về huấn luyện mô hình YOLOv8, mà còn rèn luyện khả năng xây dựng một hệ thống phần mềm hoàn chỉnh gồm Backend, Frontend, Database, xác thực người dùng, phân quyền, WebSocket, quản lý camera và báo cáo dữ liệu.

1.3. Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống web có khả năng phát hiện, quản lý và thống kê các trường hợp người tham gia giao thông không đội mũ bảo hiểm dựa trên mô hình YOLOv8.

Các mục tiêu cụ thể bao gồm:

- Tìm hiểu cơ sở lý thuyết về Computer Vision, Object Detection, Deep Learning, CNN và YOLO.
- Chuẩn bị môi trường phát triển và huấn luyện mô hình trên máy cá nhân có hỗ trợ GPU.
- Tìm kiếm, phân tích và sử dụng dataset phát hiện mũ bảo hiểm ở định dạng YOLO.
- Huấn luyện và đánh giá các mô hình YOLOv8n, YOLOv8s trên tập train, validation và test.
- So sánh các phiên bản mô hình để lựa chọn trọng số phù hợp nhất cho hệ thống.
- Xây dựng Backend bằng FastAPI để phục vụ AI inference và các nghiệp vụ hệ thống.
- Xây dựng Frontend bằng React để người dùng có thể tương tác với hệ thống thông qua giao diện web.
- Xây dựng chức năng giám sát camera thời gian thực.
- Lưu trữ ảnh bằng chứng vi phạm và thông tin detection.
- Bổ sung quy trình kiểm duyệt vi phạm để người dùng xác nhận hoặc từ chối kết quả AI.
- Xây dựng hệ thống thống kê, báo cáo và xuất dữ liệu.
- Quản lý người dùng, phân quyền Admin và Guard.
- Quản lý nhiều nguồn camera như webcam, RTSP camera và video demo.
- Ghi Audit Log cho các thao tác quan trọng.

- Chuẩn hóa cơ sở dữ liệu quan hệ từ SQLite sang PostgreSQL và quản lý schema bằng Alembic.

Thông qua các mục tiêu trên, đề tài hướng tới việc xây dựng một hệ thống không chỉ có khả năng nhận diện bằng AI, mà còn có đầy đủ các thành phần cần thiết của một ứng dụng web thực tế.

1.4. Phạm vi thực hiện

Trong phạm vi môn Thực tập cơ sở, hệ thống được xây dựng với các chức năng chính sau:

1.4.1. Phạm vi về mô hình AI

Hệ thống tập trung vào bài toán phát hiện hai lớp đối tượng:

- With Helmet: đối tượng có đội mũ bảo hiểm.
- Without Helmet: đối tượng không đội mũ bảo hiểm, được xem là trường hợp vi phạm.

Trong quá trình thực hiện, em đã tiến hành:

- Tìm kiếm dataset phù hợp từ Roboflow.
- Kiểm tra cấu trúc dữ liệu theo định dạng YOLO.
- Huấn luyện mô hình YOLOv8n để làm baseline.
- Huấn luyện mô hình YOLOv8s để cải thiện độ chính xác.
- Tiếp tục huấn luyện YOLOv8s với số epoch lớn hơn và Early Stopping.
- Đánh giá mô hình bằng các chỉ số Precision, Recall, mAP@0.5 và mAP@0.5:0.95.
- Lựa chọn mô hình YOLOv8s làm lõi nhận diện chính của hệ thống.

1.4.2. Phạm vi về Backend

Backend được xây dựng bằng FastAPI, đảm nhiệm các chức năng:

- Load và chạy mô hình YOLO.
- Xử lý ảnh upload.
- Xử lý livestream từ camera hoặc video demo.
- Quản lý camera.
- Quản lý dữ liệu vi phạm.

- Quản lý người dùng.
- Xác thực và phân quyền.
- Gửi email xác thực và reset mật khẩu.
- Xuất báo cáo Excel.
- Xuất AI Feedback Dataset.
- Ghi Audit Log.
- Cung cấp API cho Frontend.
- Gửi dữ liệu realtime qua WebSocket.

1.4.3. Phạm vi về Frontend

Frontend được xây dựng bằng React/Vite, cung cấp giao diện cho người dùng sử dụng hệ thống.

Các trang chính bao gồm:

- Login.
- Verify Email.
- Reset Password.
- Live Monitoring.
- Violation History.
- Analytics.
- User Management.
- Camera Management.
- Audit Logs.
- Profile.

Frontend hỗ trợ phân quyền theo vai trò, hiển thị livestream, nhận sự kiện realtime, kiểm duyệt vi phạm, xem ảnh bằng chứng, xuất dữ liệu và quản trị hệ thống.

1.4.4. Phạm vi về lưu trữ dữ liệu

Hệ thống sử dụng kết hợp nhiều loại lưu trữ:

- PostgreSQL: lưu dữ liệu quan hệ như user, refresh token, camera, alert, audit log.
- MongoDB: lưu dữ liệu vi phạm, detection, review metadata và traffic statistics.

- Cloudinary: lưu ảnh bằng chứng vi phạm và ảnh đại diện người dùng.
- Local static files: lưu video demo và một số tài nguyên tĩnh phục vụ hệ thống.

Việc tách dữ liệu thành nhiều nhóm giúp hệ thống phù hợp hơn với từng loại dữ liệu: dữ liệu quan hệ được quản lý bằng SQL, còn dữ liệu AI có cấu trúc linh hoạt được lưu bằng MongoDB.

1.5. Quá trình thực hiện đề tài

Quá trình thực hiện đề tài được triển khai theo từng giai đoạn qua các tuần.

Ở giai đoạn đầu, em tập trung tìm hiểu lý thuyết nền tảng như Computer Vision, Object Detection, Deep Learning, CNN, YOLOv8 và các chỉ số đánh giá mô hình. Đồng thời, em cài đặt môi trường phát triển trên máy cá nhân, cấu hình Python, PyTorch, CUDA, Ultralytics YOLO và kiểm tra khả năng sử dụng GPU.

Sau đó, em tiến hành tìm kiếm dataset, phân tích cấu trúc dữ liệu và huấn luyện các mô hình YOLOv8n, YOLOv8s. Quá trình huấn luyện được thực hiện theo nhiều bước: sanity check, debug training, final training và đánh giá trên tập validation/test. Qua quá trình so sánh, mô hình YOLOv8s cho kết quả tốt hơn YOLOv8n, đặc biệt ở lớp “Without Helmet”, nên được lựa chọn làm mô hình chính.

Ở giai đoạn tiếp theo, em xây dựng Backend bằng FastAPI để đưa mô hình AI vào hệ thống web. Ban đầu hệ thống hỗ trợ upload ảnh và trả về kết quả detection. Sau đó, hệ thống được mở rộng sang livestream camera thời gian thực, lưu ảnh bằng chứng lên Cloudinary, lưu dữ liệu vi phạm vào MongoDB và hiển thị kết quả trên Frontend.

Khi hệ thống phát triển lớn hơn, em tiếp tục bổ sung các chức năng nâng cao như xác thực người dùng, phân quyền Admin/Guard, gửi email xác thực, reset mật khẩu, thống kê báo cáo, export Excel, WebSocket, Camera Management, Demo Mode, Audit Log và Review Workflow. Đặc biệt, hệ thống được chuyển từ cơ chế “AI phát hiện là vi phạm chính thức” sang cơ chế “AI phát hiện → người dùng kiểm duyệt → confirmed/rejected”, giúp dữ liệu báo cáo đáng tin cậy hơn.

Ở giai đoạn cuối, hệ thống được chuẩn hóa về mặt hạ tầng dữ liệu bằng cách chuyển relational database từ SQLite sang PostgreSQL, tích hợp Alembic để quản lý migration schema và chuẩn hóa timestamp theo timezone-aware datetime. Đây là bước quan trọng giúp hệ thống tiến gần hơn tới môi trường triển khai thực tế.

1.6. Ý nghĩa của đề tài

Đề tài có ý nghĩa ở cả hai khía cạnh: học thuật và thực tiễn.

Về mặt học thuật, đề tài giúp em củng cố và vận dụng nhiều kiến thức quan trọng:

- Thị giác máy tính và phát hiện vật thể.
- Huấn luyện và đánh giá mô hình YOLO.
- Xử lý ảnh và video bằng OpenCV.
- Thiết kế Backend API bằng FastAPI.
- Xây dựng Frontend bằng React.
- Tổ chức cơ sở dữ liệu SQL và NoSQL.
- Xác thực, phân quyền và bảo mật cơ bản.
- Xử lý realtime bằng WebSocket.
- Quản lý migration database bằng Alembic.
- Thiết kế hệ thống phần mềm theo hướng module hóa.

Về mặt thực tiễn, hệ thống có thể đóng vai trò như một nguyên mẫu cho bài toán giám sát giao thông thông minh. Mặc dù vẫn cần tiếp tục cải thiện để triển khai thực tế ở quy mô lớn, hệ thống đã thể hiện được đầy đủ luồng xử lý từ camera, AI detection, lưu bằng chứng, kiểm duyệt, thống kê và quản trị.

2. Cơ sở lý thuyết và công nghệ sử dụng

2.1. Thị giác máy tính

Thị giác máy tính (Computer Vision) là một lĩnh vực của trí tuệ nhân tạo, tập trung vào việc giúp máy tính có khả năng thu nhận, xử lý và hiểu thông tin từ hình ảnh hoặc video. Nếu con người có thể quan sát một bức ảnh và nhận biết các đối tượng trong đó, thì Computer Vision hướng tới việc xây dựng các thuật toán giúp máy tính thực hiện các tác vụ tương tự.

Trong đề tài này, Computer Vision được ứng dụng để phân tích hình ảnh người tham gia giao thông và phát hiện xem đối tượng có đội mũ bảo hiểm hay không. Dữ liệu đầu vào của hệ thống có thể đến từ:

- Ảnh upload từ người dùng.
- Luồng livestream thời gian thực.
 - Webcam.

- Camera RTSP.

- Video demo.

Đầu ra của hệ thống là danh sách các đối tượng được phát hiện, bao gồm nhãn lớp, độ tin cậy và vị trí bounding box trên ảnh.

Bài toán này phù hợp với Computer Vision vì nó yêu cầu hệ thống không chỉ phân loại toàn bộ ảnh, mà còn phải xác định vị trí cụ thể của từng đối tượng trong ảnh. Do đó, bài toán được mô hình hóa dưới dạng Object Detection.

2.2. Bài toán Object Detection

Object Detection là bài toán phát hiện và định vị đối tượng trong ảnh. Khác với Image Classification, trong đó mô hình chỉ dự đoán ảnh thuộc lớp nào, Object Detection phải trả lời hai câu hỏi:

- Trong ảnh có những đối tượng nào?
- Mỗi đối tượng nằm ở vị trí nào?

Kết quả đầu ra của mô hình Object Detection thường bao gồm:

- Class ID: mã lớp của đối tượng.
- Class Name: tên lớp, ví dụ With Helmet hoặc Without Helmet.
- Confidence: độ tin cậy của mô hình đối với dự đoán.
- Bounding Box: khung bao quanh đối tượng, thường được biểu diễn bằng tọa độ $x1, y1, x2, y2$.

Trong đề tài này, bài toán được xây dựng với hai lớp:

- With Helmet: đối tượng có đội mũ bảo hiểm.
- Without Helmet: đối tượng không đội mũ bảo hiểm.

Việc phát hiện chính xác lớp Without Helmet là quan trọng nhất vì đây là lớp đại diện cho hành vi vi phạm. Tuy nhiên, lớp này cũng khó hơn do dữ liệu có thể bị ảnh hưởng bởi nhiều yếu tố như góc quay, ánh sáng, khoảng cách camera, chất lượng ảnh thấp hoặc đối tượng bị che khuất.

2.3. Học sâu và mạng nơ-ron tích chập

Học sâu (Deep Learning) là một nhánh của Machine Learning, sử dụng các mạng nơ-ron nhiều lớp để học biểu diễn dữ liệu ở nhiều mức độ trừu tượng khác nhau. Trong lĩnh vực xử lý ảnh, mạng nơ-ron tích chập (Convolutional Neural Network - CNN) là một

kiến trúc quan trọng vì có khả năng khai thác các đặc trưng không gian trong ảnh thông qua các phép tích chập.

Các lớp tích chập ở tầng đầu thường học các đặc trưng đơn giản như cạnh, đường nét và màu sắc. Các tầng sâu hơn có thể học các đặc trưng phức tạp hơn liên quan đến hình dạng đối tượng hoặc ngữ cảnh xuất hiện của đối tượng. Đối với bài toán phát hiện mũ bảo hiểm, cơ chế này giúp mô hình học được sự khác biệt giữa vùng đầu có mũ bảo hiểm và vùng đầu không có mũ bảo hiểm.

Tuy nhiên, nếu chỉ sử dụng CNN cho bài toán phân loại ảnh thì hệ thống không xác định được vị trí cụ thể của từng đối tượng. Vì vậy, đề tài sử dụng mô hình Object Detection như YOLO, trong đó mạng nơ-ron vừa trích xuất đặc trưng ảnh, vừa dự đoán bounding box và nhãn lớp của đối tượng.

2.4. Mô hình YOLO

YOLO, viết tắt của “You Only Look Once”, là một hướng tiếp cận phát hiện vật thể thời gian thực. Trong YOLO, bài toán phát hiện vật thể được mô hình hóa như một bài toán hồi quy trực tiếp từ ảnh đầu vào sang bounding box và xác suất lớp. Mô hình chỉ cần xử lý toàn bộ ảnh trong một lần suy luận để đưa ra kết quả phát hiện.

Ưu điểm quan trọng của YOLO là tốc độ xử lý nhanh, phù hợp với các bài toán yêu cầu thời gian thực như giám sát camera. Theo paper YOLO gốc, mô hình YOLO xử lý ảnh theo một pipeline thống nhất và có thể đạt tốc độ thời gian thực. Đây là lý do đề tài lựa chọn YOLO làm mô hình nền tảng cho bài toán phát hiện người không đội mũ bảo hiểm.

Trong đề tài, em sử dụng YOLOv8 thông qua thư viện Ultralytics. Theo tài liệu Ultralytics, YOLOv8 hỗ trợ nhiều tác vụ thị giác máy tính như object detection, instance segmentation, pose estimation, classification và oriented object detection. Đối với đề tài này, em sử dụng tác vụ object detection để phát hiện hai lớp With Helmet và Without Helmet.

Các phiên bản mô hình đã được thử nghiệm gồm:

- YOLOv8n (Nano): kích thước nhỏ, tốc độ nhanh, phù hợp làm mô hình baseline.
- YOLOv8s (Small): kích thước lớn hơn, khả năng học đặc trưng tốt hơn, cho độ chính xác cao hơn trong thực nghiệm của đề tài.

Qua quá trình huấn luyện và đánh giá trên dataset của đề tài, YOLOv8s cho kết quả tốt hơn YOLOv8n, đặc biệt ở lớp Without Helmet, nên được lựa chọn làm mô hình chính cho hệ thống.

2.5. Dataset và định dạng dữ liệu YOLO

Dataset sử dụng trong đề tài được lấy từ Roboflow Universe, thuộc project phát hiện mũ bảo hiểm. Dataset được tổ chức theo định dạng YOLO với các tập:

- train: dùng để huấn luyện mô hình.
- valid: dùng để đánh giá trong quá trình huấn luyện.
- test: dùng để kiểm tra khách quan sau khi chốt mô hình.

File cấu hình dataset có dạng data.yaml, trong đó khai báo đường dẫn tới các tập dữ liệu, số lượng class và tên class.

Trong định dạng YOLO, mỗi ảnh sẽ có một file label tương ứng. Mỗi dòng trong file label biểu diễn một đối tượng, gồm:

- Class ID.
- Tọa độ tâm bounding box theo trục x.
- Tọa độ tâm bounding box theo trục y.
- Chiều rộng bounding box.
- Chiều cao bounding box.

Các giá trị tọa độ được chuẩn hóa về khoảng từ 0 đến 1. Cách biểu diễn này giúp mô hình có thể học trên các ảnh có kích thước khác nhau, đồng thời thuận tiện cho quá trình huấn luyện bằng thư viện Ultralytics.

Trong quá trình phân tích dataset, em nhận thấy lớp With Helmet có số lượng nhãn nhiều hơn lớp Without Helmet. Đây là hiện tượng mất cân bằng dữ liệu. Vì lớp Without Helmet là lớp quan trọng đối với bài toán vi phạm, quá trình đánh giá mô hình cần đặc biệt chú ý đến Recall và mAP của lớp này.

2.6. Huấn luyện mô hình và Transfer Learning

Trong đề tài, mô hình được huấn luyện dựa trên kỹ thuật Transfer Learning. Thay vì huấn luyện mô hình từ đầu với trọng số ngẫu nhiên, em sử dụng các trọng số pretrained như yolov8n.pt và yolov8s.pt.

Transfer Learning giúp quá trình huấn luyện hiệu quả hơn vì mô hình đã học được nhiều đặc trưng thị giác cơ bản từ tập dữ liệu lớn trước đó. Khi đưa vào bài toán phát hiện mũ bảo hiểm, mô hình chỉ cần tinh chỉnh lại trọng số để phù hợp với hai lớp With Helmet và Without Helmet.

Quy trình huấn luyện được thực hiện theo nhiều giai đoạn:

- Sanity Check: huấn luyện nhanh với số epoch rất nhỏ để kiểm tra dataset, pipeline và môi trường.
- Debug Training: huấn luyện với số epoch trung bình để kiểm tra khả năng học của mô hình.
- Final Training: huấn luyện chính thức với số epoch lớn hơn.
- Extended Training: tiếp tục huấn luyện YOLOv8s với số epoch lớn hơn và Early Stopping để tìm trọng số tốt hơn.

Việc chia quá trình huấn luyện thành nhiều giai đoạn giúp hạn chế rủi ro. Nếu dataset có lỗi, cấu trúc thư mục sai hoặc GPU không hoạt động, các vấn đề sẽ được phát hiện sớm ở giai đoạn sanity check thay vì tốn nhiều thời gian huấn luyện chính thức.

2.7. Các chỉ số đánh giá mô hình

Để đánh giá mô hình Object Detection, đề tài sử dụng các chỉ số phổ biến như Precision, Recall, IoU, AP và mAP.

2.7.1. Precision

Precision đo tỷ lệ dự đoán đúng trong số các dự đoán mà mô hình đưa ra.

Trong hệ thống phát hiện vi phạm, Precision cao có nghĩa là khi mô hình báo một trường hợp không đội mũ, khả năng đó là vi phạm thật cao. Điều này giúp giảm cảnh báo sai.

Nếu Precision thấp, hệ thống sẽ sinh ra nhiều false positive, khiến người dùng phải kiểm tra nhiều bản ghi không hợp lệ.

2.7.2. Recall

Recall đo tỷ lệ phát hiện đúng trong số các đối tượng thật sự tồn tại.

Trong bài toán này, Recall của lớp Without Helmet rất quan trọng. Recall thấp có nghĩa là hệ thống bỏ sót nhiều người không đội mũ bảo hiểm. Đây là vấn đề nghiêm trọng vì mục tiêu chính của hệ thống là phát hiện vi phạm.

Do đó, khi đánh giá mô hình, cần cân bằng giữa Precision và Recall. Một mô hình chỉ có Precision cao nhưng Recall thấp sẽ ít báo sai nhưng lại bỏ sót nhiều vi phạm. Ngược lại, Recall cao nhưng Precision thấp sẽ phát hiện nhiều nhưng tạo nhiều cảnh báo sai.

2.7.3. IoU

IoU là viết tắt của Intersection over Union, dùng để đo mức độ trùng khớp giữa bounding box dự đoán và bounding box thực tế.

IoU càng cao thì bounding box dự đoán càng sát với đối tượng thật. Trong hệ thống livestream, bounding box chính xác giúp giao diện hiển thị trực quan hơn và hỗ trợ quá trình tracking ổn định hơn.

2.7.4. mAP

mAP là chỉ số tổng hợp quan trọng trong Object Detection.

Trong báo cáo, em sử dụng hai chỉ số chính:

- $mAP@0.5$: đánh giá khi ngưỡng IoU là 0.5.
- $mAP@0.5:0.95$: đánh giá nghiêm ngặt hơn với nhiều ngưỡng IoU từ 0.5 đến 0.95.

$mAP@0.5$ cho biết khả năng phát hiện tổng quát của mô hình, còn $mAP@0.5:0.95$ phản ánh chất lượng định vị bounding box khắt khe hơn

2.8. Tracking và ByteTrack trong video realtime

Trong xử lý video thời gian thực, việc phát hiện vật thể trên từng frame riêng lẻ là chưa đủ, vì cùng một đối tượng có thể xuất hiện liên tục trong nhiều frame. Nếu hệ thống chỉ dựa vào kết quả detection độc lập, cùng một người có thể bị ghi nhận nhiều lần hoặc bị mất dấu khi đối tượng bị che khuất tạm thời. Vì vậy, các hệ thống giám sát video thường cần kết hợp Object Detection với Multi-Object Tracking.

Multi-Object Tracking là bài toán theo dõi nhiều đối tượng qua các frame liên tiếp, đồng thời gán cho mỗi đối tượng một mã định danh riêng. Nhờ đó, hệ thống có thể biết

được đối tượng ở frame hiện tại có phải là cùng một đối tượng đã xuất hiện ở các frame trước hay không.

Trong đề tài, em sử dụng ByteTrack để hỗ trợ quá trình tracking. ByteTrack là một phương pháp Multi-Object Tracking dựa trên việc liên kết các detection box qua nhiều frame. Điểm nổi bật của ByteTrack là không chỉ sử dụng các detection có độ tin cậy cao, mà còn tận dụng cả các detection có độ tin cậy thấp để khôi phục các đối tượng bị che khuất hoặc khó phát hiện, từ đó cải thiện khả năng duy trì track ID.

Việc sử dụng tracking trong hệ thống giúp hạn chế ghi trùng vi phạm, theo dõi đối tượng ổn định hơn trong luồng video và hỗ trợ quá trình xác nhận vi phạm dựa trên nhiều frame thay vì chỉ dựa vào một dự đoán đơn lẻ. Đây là yếu tố quan trọng đối với hệ thống giám sát thời gian thực, vì kết quả nhận diện trên một frame có thể bị ảnh hưởng bởi ánh sáng, chuyển động, góc quay hoặc hiện tượng che khuất.

2.9. Xử lý ảnh và video bằng OpenCV

OpenCV là thư viện mã nguồn mở được sử dụng phổ biến trong các bài toán xử lý ảnh và thị giác máy tính. Theo tài liệu chính thức, OpenCV cung cấp nhiều chức năng phục vụ xử lý ảnh, video, camera và các tác vụ Computer Vision trong thời gian thực.

Trong đề tài, OpenCV được sử dụng để hỗ trợ quá trình đọc và xử lý dữ liệu hình ảnh trước và sau khi đưa vào mô hình AI. Cụ thể, OpenCV có thể đọc ảnh từ file upload, chuyển đổi dữ liệu ảnh sang dạng mảng, mở luồng video từ webcam, camera RTSP hoặc video file thông qua VideoCapture, đồng thời trích xuất từng frame để phục vụ quá trình nhận diện.

Bên cạnh đó, OpenCV cũng hỗ trợ các thao tác trực quan hóa kết quả như vẽ bounding box, hiển thị nhãn lớp, confidence và tracking ID lên khung hình. Sau khi xử lý, frame có thể được mã hóa lại thành định dạng ảnh phù hợp để truyền về Frontend trong luồng livestream.

Việc sử dụng OpenCV giúp hệ thống có thể kết nối giữa dữ liệu ảnh/video đầu vào và mô hình YOLOv8, đồng thời hỗ trợ hiển thị kết quả nhận diện một cách trực quan trên giao diện giám sát.

2.10. Streaming video thời gian thực

Streaming video thời gian thực là kỹ thuật truyền dữ liệu hình ảnh liên tục từ nguồn phát, như camera hoặc video, đến phía người dùng để hiển thị gần như ngay lập tức. Trong các hệ thống giám sát, streaming đóng vai trò quan trọng vì người dùng cần quan sát trực tiếp tình trạng hiện tại của khu vực được theo dõi.

Trong đề tài, luồng video được truyền từ Backend tới Frontend theo cơ chế MJPEG Streaming. Về cơ bản, MJPEG gửi liên tục các frame ảnh JPEG thông qua một HTTP response dạng stream. Cách tiếp cận này tương đối đơn giản, dễ triển khai với các web framework như FastAPI và phù hợp với hệ thống giám sát cần hiển thị hình ảnh camera trực tiếp.

2.11. Backend FastAPI

FastAPI là framework được sử dụng để xây dựng tầng Backend của hệ thống. Theo tài liệu chính thức, FastAPI là một web framework hiện đại, có hiệu năng cao, dùng để xây dựng API với Python dựa trên hệ thống type hints chuẩn của Python.

FastAPI phù hợp với đề tài vì hỗ trợ xây dựng REST API nhanh, có khả năng xử lý bất đồng bộ thông qua `async/await`, đồng thời tích hợp tốt với Pydantic để kiểm tra và chuẩn hóa dữ liệu request/response. Điều này giúp quá trình trao đổi dữ liệu giữa Frontend và Backend rõ ràng, hạn chế lỗi sai kiểu dữ liệu và hỗ trợ sinh tài liệu API tự động.

Bên cạnh REST API, FastAPI còn hỗ trợ WebSocket, phù hợp với các hệ thống cần cập nhật dữ liệu thời gian thực như livestream, thông báo vi phạm mới, trạng thái camera và các sự kiện hệ thống. Ngoài ra, cơ chế Dependency Injection của FastAPI giúp tái sử dụng các logic chung như xác thực người dùng, phân quyền, lấy kết nối database hoặc lấy model AI.

Với các đặc điểm trên, FastAPI là lựa chọn phù hợp để xây dựng Backend cho hệ thống web phát hiện người không đội mũ bảo hiểm, đặc biệt trong bối cảnh hệ thống cần kết hợp API, AI inference, xử lý bất đồng bộ và dữ liệu realtime.

2.12. Frontend React/Vite

Frontend của hệ thống được xây dựng bằng React kết hợp với Vite. Theo tài liệu chính thức, React là thư viện JavaScript dùng để xây dựng giao diện người dùng dựa trên

các component độc lập, giúp lập trình viên có thể chia nhỏ giao diện thành các phần dễ quản lý và tái sử dụng.

React phù hợp với đề tài vì hệ thống có nhiều màn hình và nhiều thành phần giao diện lặp lại như bảng dữ liệu, modal, form, bộ lọc, pagination, dropdown và các thẻ thống kê. Cách tiếp cận theo component giúp mã nguồn Frontend dễ tổ chức, dễ bảo trì và thuận tiện khi mở rộng thêm chức năng mới.

Vite được sử dụng làm công cụ build và môi trường phát triển cho Frontend. Vite hỗ trợ khởi động dự án nhanh, cập nhật giao diện gần như tức thời trong quá trình phát triển và tối ưu quá trình build ứng dụng web. Điều này phù hợp với quá trình xây dựng hệ thống Dashboard có nhiều trang và thường xuyên cần chỉnh sửa giao diện.

Nhờ sự kết hợp giữa React và Vite, Frontend của hệ thống có thể được phát triển nhanh, giao diện được tổ chức rõ ràng theo component và dễ tích hợp với Backend thông qua REST API, WebSocket và các cơ chế xác thực người dùng.

2.13. Cơ sở dữ liệu PostgreSQL và SQLAlchemy

Hệ thống sử dụng PostgreSQL làm cơ sở dữ liệu quan hệ chính. PostgreSQL được dùng để lưu các dữ liệu có cấu trúc rõ ràng, có quan hệ và cần tính nhất quán cao.

SQLModel được sử dụng làm ORM để định nghĩa model và thao tác với database. SQLAlchemy kết hợp giữa SQLAlchemy và Pydantic, giúp model vừa có thể ánh xạ xuống database, vừa có thể tận dụng khả năng kiểm tra kiểu dữ liệu.

Ban đầu hệ thống sử dụng SQLite trong giai đoạn phát triển. Tuy nhiên, ở giai đoạn sau, hệ thống được chuyển sang PostgreSQL để phù hợp hơn với định hướng production. Việc chuyển đổi này giúp hệ thống có nền tảng dữ liệu ổn định hơn và phù hợp hơn với ứng dụng backend nhiều người dùng.

2.14. Alembic và quản lý migration database

Alembic được sử dụng để quản lý schema migration cho PostgreSQL.

Trước đây, hệ thống có thể dùng SQLModel.metadata.create_all() để tự tạo bảng khi ứng dụng khởi động. Cách này tiện trong giai đoạn đầu, nhưng không phù hợp với hệ thống lớn vì khó kiểm soát lịch sử thay đổi schema.

Với Alembic, mỗi thay đổi về database schema được lưu thành một migration file. Điều này giúp:

- Theo dõi lịch sử thay đổi schema.
- Dựng database mới một cách nhất quán.
- Upgrade hoặc downgrade database khi cần.
- Phù hợp hơn với môi trường production.
- Tránh việc ứng dụng tự ý thay đổi schema khi startup.

2.15. MongoDB và dữ liệu AI

MongoDB là hệ quản trị cơ sở dữ liệu NoSQL theo mô hình document. Theo tài liệu chính thức, MongoDB lưu dữ liệu dưới dạng BSON document, tức là dạng biểu diễn nhị phân của JSON document và hỗ trợ nhiều kiểu dữ liệu khác nhau.

Trong đề tài, MongoDB được sử dụng để lưu các dữ liệu liên quan đến AI như kết quả nhận diện, bounding box, confidence, ảnh bằng chứng, thông tin camera và trạng thái kiểm duyệt. Các dữ liệu này có cấu trúc linh hoạt, có thể thay đổi hoặc mở rộng trong quá trình phát triển, nên phù hợp với mô hình lưu trữ document của MongoDB.

Việc sử dụng MongoDB giúp hệ thống dễ dàng lưu trữ các bản ghi vi phạm và dữ liệu thống kê mà không cần thiết kế quá nhiều bảng quan hệ phức tạp như khi dùng cơ sở dữ liệu SQL.

2.16. Cloudinary và lưu trữ ảnh bằng chứng

Cloudinary được sử dụng để lưu ảnh bằng chứng vi phạm và ảnh đại diện người dùng.

Khi hệ thống phát hiện một vi phạm hợp lệ, ảnh bằng chứng được upload lên Cloudinary. MongoDB chỉ lưu URL ảnh và metadata liên quan. Cách làm này có một số lợi ích:

- Giảm tải lưu trữ trên server Backend.
- Dễ hiển thị ảnh từ Frontend.
- Dễ quản lý ảnh bằng chứng.
- Phù hợp hơn khi triển khai hệ thống trên nhiều môi trường.

Trong quá trình xử lý realtime, việc upload ảnh có thể tốn thời gian, vì vậy hệ thống sử dụng cơ chế xử lý bất đồng bộ để hạn chế ảnh hưởng đến luồng livestream.

2.17. Xác thực, phân quyền và bảo mật

Xác thực và phân quyền là thành phần quan trọng trong các hệ thống web có nhiều người dùng. Trong đề tài, các cơ chế bảo mật được xây dựng dựa trên các công nghệ phổ biến như password hashing, JSON Web Token, refresh token và phân quyền theo vai trò.

Đối với mật khẩu người dùng, hệ thống không lưu mật khẩu dạng văn bản thuần mà sử dụng cơ chế băm mật khẩu trước khi lưu vào cơ sở dữ liệu. Cách làm này giúp giảm rủi ro lộ mật khẩu thật nếu dữ liệu bị truy cập trái phép.

Sau khi người dùng đăng nhập thành công, hệ thống sử dụng JSON Web Token (JWT) để xác thực các request tiếp theo. Theo RFC 7519, JWT là một phương thức compact và URL-safe để biểu diễn các claims giữa hai bên, đồng thời có thể được ký số để đảm bảo tính toàn vẹn của dữ liệu. Access token thường có thời hạn ngắn và được gửi kèm trong các request cần xác thực.

Bên cạnh access token, hệ thống sử dụng refresh token để cấp lại access token khi token cũ hết hạn. Refresh token giúp duy trì phiên đăng nhập mà không yêu cầu người dùng đăng nhập lại quá thường xuyên. Trong các hệ thống web, refresh token thường được bảo vệ kỹ hơn access token, ví dụ lưu trong cookie httpOnly để hạn chế việc truy cập trực tiếp từ JavaScript phía client.

Ngoài xác thực, hệ thống còn sử dụng cơ chế phân quyền theo vai trò (Role-Based Access Control - RBAC). Với RBAC, quyền truy cập được xác định dựa trên vai trò của người dùng thay vì xử lý rời rạc cho từng tài khoản. Cách tiếp cận này giúp hệ thống dễ quản lý quyền hơn khi số lượng người dùng và chức năng tăng lên.

Nhìn chung, việc kết hợp password hashing, JWT, refresh token, cookie httpOnly và RBAC giúp hệ thống có nền tảng bảo mật phù hợp cho một ứng dụng web có đăng nhập, phân quyền và nhiều chức năng quản trị.

2.18. WebSocket và dữ liệu realtime

WebSocket là giao thức hỗ trợ giao tiếp hai chiều giữa client và server trên một kết nối duy nhất. Theo RFC 6455, sau khi quá trình handshake hoàn tất, WebSocket cho phép hai bên gửi dữ liệu qua lại mà không cần liên tục tạo các request HTTP mới.

Trong các ứng dụng web thông thường, client thường phải gửi request định kỳ để lấy dữ liệu mới từ server. Cách làm này gọi là polling và có thể gây lãng phí tài nguyên nếu dữ liệu cần cập nhật thường xuyên. WebSocket giải quyết vấn đề này bằng cách duy

tri kết nối lâu dài, cho phép server chủ động gửi dữ liệu mới đến client ngay khi có sự kiện phát sinh.

WebSocket phù hợp với các hệ thống cần cập nhật dữ liệu theo thời gian thực như dashboard giám sát, thông báo sự kiện, trạng thái kết nối, dữ liệu cảm biến hoặc các hệ thống livestream có kèm thông tin trạng thái. Trong đề tài, WebSocket được sử dụng để hỗ trợ việc cập nhật các dữ liệu realtime bên cạnh luồng hình ảnh, giúp giao diện người dùng phản hồi nhanh hơn mà không cần tải lại trang.

Việc sử dụng WebSocket giúp giảm số lượng request lặp lại, cải thiện độ trễ cập nhật dữ liệu và phù hợp với yêu cầu của một hệ thống giám sát thông minh có nhiều sự kiện phát sinh liên tục.

2.19. Gửi Email và SMTP

Trong hệ thống, email được sử dụng cho các chức năng liên quan đến xác thực tài khoản và khôi phục mật khẩu. Cụ thể, khi người dùng được tạo tài khoản hoặc yêu cầu đặt lại mật khẩu, hệ thống gửi email chứa đường dẫn xác thực hoặc token reset password đến địa chỉ email của người dùng.

Về mặt kỹ thuật, hệ thống sử dụng giao thức SMTP để gửi email từ Backend tới máy chủ email. Theo RFC 5321, mục tiêu của SMTP là truyền tải thư điện tử một cách tin cậy và hiệu quả. Trong đề tài, Backend đóng vai trò SMTP client, kết nối tới SMTP server đã cấu hình trong file môi trường và gửi email có nội dung HTML tới người dùng.

Các trường hợp sử dụng email trong hệ thống bao gồm:

- Gửi email xác thực tài khoản sau khi tạo người dùng.
- Gửi email đặt lại mật khẩu khi người dùng quên mật khẩu.

Nội dung email được xây dựng bằng HTML template thay vì chỉ gửi văn bản thuần. Cách làm này giúp email có giao diện rõ ràng hơn và phù hợp với trải nghiệm người dùng của một hệ thống web hoàn chỉnh.

Trong quá trình triển khai, thao tác gửi email được đưa vào background task để không làm chậm phản hồi API chính. Nhờ đó, khi hệ thống tạo tài khoản hoặc xử lý yêu cầu quên mật khẩu, người dùng không phải chờ quá lâu trong lúc Backend kết nối SMTP server và gửi email.

2.20. Các công nghệ chính sử dụng trong hệ thống

Tổng hợp các công nghệ chính được sử dụng:

Nhóm công nghệ	Công nghệ sử dụng	Vai trò
AI/Computer Vision	YOLOv8, Ultralytics, OpenCV, NumPy, ByteTrack	Phát hiện, tracking và xử lý ảnh/video
Backend	FastAPI, Uvicorn, Pydantic, SQLAlchemy	Xây dựng API, xử lý nghiệp vụ và quản lý dữ liệu
Frontend	React, Vite, React Router, Tailwind CSS, Recharts	Xây dựng giao diện web
SQL Database	PostgreSQL, Alembic	Lưu dữ liệu quan hệ và quản lý migration
NoSQL Database	MongoDB, Motor	Lưu dữ liệu violation và traffic statistics
Cloud Storage	Cloudinary	Lưu ảnh bằng chứng và avatar
Authentication	JWT, Refresh Token, pwdlib/argon2	Xác thực và bảo mật người dùng
Realtime	WebSocket, MJPEG Streaming	Cập nhật sự kiện và hiển thị camera realtime
Export/Report	openpyxl, csv, zipfile, json	Xuất báo cáo Excel và AI Feedback Dataset
Email	SMTP, HTML template	Xác thực email và reset mật khẩu

2.21. Nhận xét về lựa chọn công nghệ

Các công nghệ được lựa chọn phù hợp với mục tiêu của đề tài. YOLOv8 phù hợp với bài toán phát hiện vật thể thời gian thực vì được Ultralytics hỗ trợ tốt cho tác vụ object detection và có thể triển khai thuận tiện trong pipeline Python. FastAPI phù hợp với Backend AI vì hỗ trợ xây dựng API dựa trên type hints, có khả năng xử lý bất đồng bộ và tích hợp tốt với WebSocket. React phù hợp với Frontend Dashboard vì hỗ trợ xây dựng giao diện theo component, giúp tách nhỏ và tái sử dụng các thành phần UI.

Việc kết hợp PostgreSQL và MongoDB giúp hệ thống tận dụng được ưu điểm của cả hai loại cơ sở dữ liệu. PostgreSQL phù hợp với dữ liệu quan hệ như user, camera,

token và audit log. MongoDB phù hợp với dữ liệu AI có cấu trúc linh hoạt như detection, bounding box, review metadata và traffic statistics.

Cloudinary giúp hệ thống lưu trữ ảnh bằng chứng thông qua API upload media thay vì lưu toàn bộ ảnh trên server Backend. WebSocket hỗ trợ truyền dữ liệu realtime hai chiều giữa Backend và Frontend. SMTP được sử dụng cho các chức năng gửi email xác thực và reset mật khẩu, phù hợp với chuẩn truyền tải email được định nghĩa trong RFC 5321. Alembic giúp hệ thống quản lý schema database theo migration script, phù hợp hơn với quá trình phát triển và triển khai PostgreSQL.

Nhìn chung, bộ công nghệ được sử dụng đã đáp ứng tốt yêu cầu của đề tài, đồng thời giúp em tiếp cận được quy trình xây dựng một hệ thống AI Web Application tương đối hoàn chỉnh.

3. Phân tích và thiết kế hệ thống

3.1. Tổng quan hệ thống

Hệ thống được xây dựng nhằm hỗ trợ phát hiện, quản lý và thống kê các trường hợp người tham gia giao thông không đội mũ bảo hiểm thông qua ảnh, video hoặc camera thời gian thực. Khác với một chương trình nhận diện đơn lẻ chỉ chạy trên máy cá nhân, hệ thống trong đề tài được thiết kế theo hướng một ứng dụng web hoàn chỉnh, có đầy đủ các thành phần như Backend, Frontend, cơ sở dữ liệu, lưu trữ ảnh bằng chứng, xác thực người dùng, phân quyền, thống kê, kiểm duyệt dữ liệu và quản lý camera.

Luồng xử lý tổng quát của hệ thống có thể mô tả như sau:

1. Camera / Video / Image
2. Backend FastAPI
3. YOLOv8s Detection + Tracking
4. Xác nhận vi phạm bằng logic nhiều frame
5. Lưu ảnh bằng chứng lên Cloudinary
6. Lưu metadata vào MongoDB
7. Đẩy sự kiện realtime qua WebSocket
8. Frontend Dashboard hiển thị và cho phép người dùng kiểm duyệt

Cách thiết kế này giúp hệ thống không chỉ nhận diện vi phạm, mà còn hỗ trợ quản lý vòng đời của dữ liệu vi phạm từ lúc AI phát hiện, lưu trữ bằng chứng, kiểm duyệt, thống kê cho tới xuất báo cáo.

3.2. Phân tích yêu cầu hệ thống

3.2.1. Yêu cầu chức năng

Dựa trên mục tiêu của đề tài, hệ thống cần đáp ứng các nhóm chức năng chính sau:

3.2.1.1. Nhóm chức năng xác thực và phân quyền

Hệ thống cần có cơ chế đăng nhập và phân quyền để đảm bảo chỉ người dùng hợp lệ mới được sử dụng các chức năng nghiệp vụ.

Các chức năng bao gồm:

- Đăng nhập bằng tài khoản và mật khẩu.
- Cấp access token sau khi đăng nhập thành công.
- Cấp lại access token thông qua refresh token.
- Đăng xuất và thu hồi refresh token.
- Xác thực email người dùng.
- Gửi email đặt lại mật khẩu.
- Đặt lại mật khẩu thông qua token reset.
- Phân quyền người dùng theo vai trò Admin và Guard.
- Bảo vệ các route quan trọng ở cả Backend và Frontend.

3.2.1.2. Nhóm chức năng nhận diện ảnh

Hệ thống cần hỗ trợ người dùng upload ảnh và trả về kết quả nhận diện.

Các yêu cầu chính:

- Cho phép upload file ảnh.
- Kiểm tra định dạng file.
- Đọc ảnh và đưa vào mô hình YOLOv8s.
- Trả về danh sách detection gồm class, confidence và bounding box.
- Trả về ảnh đã được vẽ bounding box.
- Lưu vi phạm nếu phát hiện lớp Without Helmet.

3.2.1.3. Nhóm chức năng giám sát livestream

Đây là chức năng trung tâm của hệ thống.

Các yêu cầu chính:

- Cho phép bật/tắt camera từ giao diện web.
- Hiện thị livestream realtime trên Dashboard.
- Chạy mô hình AI trên các frame video.
- Vẽ bounding box, label và tracking ID lên khung hình.
- Phát hiện người không đội mũ bảo hiểm.
- Hạn chế ghi trùng cùng một vi phạm.
- Đẩy sự kiện vi phạm mới về Frontend qua WebSocket.
- Hiện thị telemetry như AI FPS và Capture FPS.
- Hỗ trợ nhiều nguồn camera như webcam, RTSP và video demo.

3.2.1.4. Nhóm chức năng quản lý vi phạm

Hệ thống cần lưu và quản lý các bản ghi vi phạm một cách có tổ chức.

Các chức năng bao gồm:

- Xem danh sách vi phạm.
- Lọc vi phạm theo ngày, trạng thái, camera hoặc số lượng vi phạm.
- Xem ảnh bằng chứng.
- Xem chi tiết detection.
- Xác nhận vi phạm.
- Từ chối vi phạm.
- Chọn lý do từ chối.
- Ghi chú khi review.
- Xóa vi phạm theo quyền Admin.
- Xuất danh sách vi phạm ra file Excel.
- Xuất AI Feedback Dataset phục vụ cải thiện mô hình.

3.2.1.5. Nhóm chức năng thống kê và báo cáo

Hệ thống cần có khả năng tổng hợp dữ liệu phục vụ phân tích.

Các chức năng gồm:

- Thống kê tổng số vi phạm.
- Thống kê số người an toàn.
- Thống kê theo ngày hoặc theo giờ.

- Hiển thị biểu đồ xu hướng.
- Theo dõi độ chính xác trung bình dựa trên confidence.
- Chỉ tính các vi phạm đã được xác nhận trong báo cáo chính thức.
- Tự động làm mới dữ liệu theo chu kỳ phù hợp.

3.2.1.6. Nhóm chức năng quản lý camera

Hệ thống cần hỗ trợ quản trị nguồn camera từ giao diện.

Các chức năng gồm:

- Tạo camera mới.
- Sửa thông tin camera.
- Bật/tắt camera.
- Xóa mềm camera.
- Test connection.
- Chọn camera trong Live Monitoring.
- Hỗ trợ camera webcam, RTSP và video file.
- Không hiển thị source URL nhạy cảm cho người dùng không cần thiết.

3.2.1.7. Nhóm chức năng Audit Log

Hệ thống cần ghi lại các thao tác quan trọng để phục vụ truy vết.

Các chức năng gồm:

- Ghi log khi tạo/sửa/xóa người dùng.
- Ghi log khi confirm/reject/delete violation.
- Ghi log khi tạo/sửa/xóa/bật/tắt camera.
- Ghi log khi export dữ liệu.
- Cho Admin xem danh sách audit log.
- Cho phép lọc và xem metadata chi tiết.

3.2.2. Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng một số yêu cầu phi chức năng.

3.2.2.1. Hiệu năng realtime

Hệ thống cần xử lý video đủ nhanh để người dùng có thể quan sát livestream mượt mà. Việc chạy AI không được làm luồng video bị giật lag quá nhiều. Vì vậy, hệ thống được thiết kế với các cơ chế như frame skipping, tracking theo nhiều frame, tách FPS capture và FPS inference, đồng thời xử lý các tác vụ nặng như upload ảnh theo hướng bất đồng bộ.

3.2.2.2. Tính ổn định

Camera có thể bị tắt, đổi nguồn hoặc mất kết nối. Do đó, hệ thống cần có cơ chế quản lý vòng đời camera, giải phóng tài nguyên khi người dùng ngắt kết nối, tránh zombie thread và tránh giữ camera handle cũ khi switch camera.

3.2.2.3. Tính bảo mật

Hệ thống có dữ liệu người dùng, token, RTSP URL và ảnh bằng chứng vi phạm. Vì vậy, hệ thống cần đảm bảo:

- Không lưu mật khẩu dạng plain text.
- Refresh token được lưu dạng hash.
- Phân quyền API theo vai trò.
- Không trả RTSP URL nhạy cảm về Frontend không cần thiết.
- Cấu hình bí mật được lưu trong file môi trường.
- Các thao tác quan trọng được ghi audit log.

3.2.2.4. Tính mở rộng

Hệ thống được thiết kế theo kiến trúc module hóa. Các chức năng như auth, camera, violation, report, audit log được tách thành các module riêng. Điều này giúp việc mở rộng thêm chức năng trong tương lai dễ dàng hơn.

3.2.2.5. Tính dễ bảo trì

Backend được tách thành các tầng router, service, model, schema, dependency và utility. Frontend được chia thành các page và component nhỏ. Cách tổ chức này giúp giảm độ phức tạp của từng file và thuận tiện hơn khi sửa lỗi hoặc bổ sung tính năng.

3.3. Nhóm người sử dụng hệ thống

Hệ thống có hai nhóm người dùng chính:

3.3.1. Admin

Admin là người có quyền quản trị cao nhất trong hệ thống.

Các quyền chính của Admin gồm:

- Quản lý người dùng.
- Quản lý camera.
- Xem báo cáo.
- Xem audit log.
- Xem livestream.
- Xem và kiểm duyệt vi phạm.
- Xóa vi phạm.
- Xuất báo cáo.
- Xuất AI Feedback Dataset.

Admin phù hợp với vai trò người quản trị hệ thống hoặc người phụ trách vận hành tổng thể.

3.3.2. Guard

Guard là người dùng vận hành hệ thống giám sát hằng ngày.

Các quyền chính gồm:

- Đăng nhập hệ thống.
- Xem livestream.
- Xem lịch sử vi phạm.
- Xác nhận hoặc từ chối vi phạm.
- Xem báo cáo.
- Theo dõi thông báo realtime.

Guard không có quyền quản trị người dùng, quản lý camera nâng cao hoặc xem audit log nếu không được cấp quyền Admin.

3.4. Thiết kế kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình client-server kết hợp với AI inference pipeline.

Kiến trúc tổng thể gồm các lớp chính:

- Frontend React
- Backend FastAPI
- AI Model
- PostgreSQL / MongoDB / Cloudinary

Trong đó:

- Frontend chịu trách nhiệm hiển thị giao diện và tương tác với người dùng.
- Backend chịu trách nhiệm xử lý nghiệp vụ, bảo mật, AI inference và kết nối dữ liệu.
- AI Model chịu trách nhiệm phát hiện đối tượng.
- PostgreSQL lưu dữ liệu quan hệ.
- MongoDB lưu dữ liệu violation và traffic statistics.
- Cloudinary lưu ảnh bằng chứng.
- WebSocket truyền dữ liệu realtime.
- MJPEG truyền luồng hình ảnh camera.

Cách thiết kế này giúp hệ thống tách rõ trách nhiệm giữa giao diện, xử lý nghiệp vụ, AI và dữ liệu. Frontend không trực tiếp truy cập database hoặc model AI, mà luôn tương tác thông qua Backend API. Điều này giúp kiểm soát bảo mật và logic nghiệp vụ tốt hơn.

3.5. Thiết kế kiến trúc Backend

3.5.1. Cấu trúc Backend

Backend được tổ chức theo hướng module hóa với các thư mục chính:

SourceCode/BE/app/

└── api/

└── services/

└── database/

```
|— models/
|— schemas/
|— core/
|— dependencies/
|— utils/
└— exceptions/
```

Ý nghĩa của từng thư mục:

- api: chứa các router định nghĩa endpoint.
- services: chứa logic nghiệp vụ chính.
- database: chứa kết nối tới các database
- models: chứa các model ánh xạ database.
- schemas: chứa các schema request/response.
- core: chứa cấu hình hệ thống, security, websocket manager.
- dependencies: chứa các dependency dùng chung cho FastAPI.
- utils: chứa các hàm tiện ích.
- exceptions: chứa exception custom và global handler.

Cách tổ chức này giúp Backend không bị dồn logic vào router. Router chỉ nhận request, kiểm tra quyền và gọi service. Service chịu trách nhiệm xử lý nghiệp vụ, truy cập database hoặc gọi các module liên quan.

3.5.2. Thiết kế xử lý AI Model

Mô hình YOLOv8s được load khi Backend khởi động và được lưu trong app.state để tái sử dụng trong suốt vòng đời ứng dụng. Cách này tránh việc load lại model ở mỗi request, giúp giảm thời gian xử lý và tiết kiệm tài nguyên GPU/CPU.

Với ảnh tĩnh, service thực hiện các bước:

1. Upload Image
2. Decode image bằng OpenCV
3. YOLOv8s predict
4. Trích xuất detection
5. Vẽ bounding box
6. Encode ảnh kết quả
7. Trả response cho Frontend

Với livestream, pipeline phức tạp hơn vì cần xử lý nhiều frame liên tục, tracking đối tượng và tránh ghi log trùng.

3.6. Thiết kế Frontend

3.6.1. Cấu trúc Frontend

Frontend được xây dựng bằng React/Vite và tổ chức theo hướng component.

Cấu trúc tổng quát gồm:

SourceCode/FE/src/

├── components/

├── pages/

├── services/

├── contexts/

├── utils/

└── assets/

Trong đó:

- pages: chứa các màn hình chính.
- components: chứa các thành phần UI tái sử dụng.
- services: chứa logic gọi API và WebSocket.
- contexts: chứa context như AuthContext.
- utils: chứa các hàm tiện ích dùng chung.
- assets: chứa tài nguyên giao diện.

3.6.2. Thiết kế các trang chính

Các trang chính của hệ thống gồm:

3.6.2.1. Login

Trang đăng nhập cho phép người dùng nhập username và password. Ngoài ra, trang này cũng hỗ trợ luồng forgot password để gửi email đặt lại mật khẩu.

3.6.2.2. Verify Email

Trang này nhận token từ URL, gọi API xác thực email và cập nhật trạng thái tài khoản. Nếu xác thực thành công, người dùng có thể truy cập các chức năng chính của hệ thống.

3.6.2.3. Reset Password

Trang reset password cho phép người dùng nhập mật khẩu mới sau khi nhận link qua email. Giao diện có kiểm tra xác nhận mật khẩu và trạng thái hợp lệ của token.

3.6.2.4. Live Monitoring

Đây là trang giám sát trực tiếp. Trang hiển thị livestream, trạng thái camera, telemetry, số vi phạm trong phiên và thông báo realtime. Người dùng có thể bật/tắt camera, chọn camera source và theo dõi detection trực tiếp.

3.6.2.5. Violation History

Trang này hiển thị danh sách vi phạm. Người dùng có thể lọc dữ liệu, xem ảnh bằng chứng, confirm/reject vi phạm, export dữ liệu và xem chi tiết detection.

3.6.2.6. Analytics

Trang Analytics hiển thị các thống kê về vi phạm, traffic statistics, accuracy và xu hướng theo thời gian.

3.6.2.7. User Management

Trang này dành cho Admin để tạo, sửa, khóa hoặc quản lý người dùng.

3.6.2.8. Camera Management

Trang này dành cho Admin để quản lý các nguồn camera, gồm webcam, RTSP và video demo.

3.6.2.9. Audit Logs

Trang này dành cho Admin để xem lịch sử các thao tác quan trọng trong hệ thống.

3.6.3. Thiết kế Route Protection

Frontend sử dụng cơ chế bảo vệ route để kiểm soát quyền truy cập.

Các trạng thái chính gồm:

- Chưa đăng nhập: chỉ được truy cập Login, Verify Email, Reset Password.
- Đã đăng nhập nhưng chưa verify email: bị hạn chế truy cập chức năng nghiệp vụ.
- Đã đăng nhập và có role Guard: được truy cập các chức năng giám sát và review.
- Đã đăng nhập và có role Admin: được truy cập thêm các trang quản trị.

Cơ chế này giúp đảm bảo người dùng chỉ nhìn thấy và sử dụng các chức năng phù hợp với quyền hạn.

3.6.4. Thiết kế giao tiếp API

Frontend giao tiếp với Backend thông qua Axios service.

Axios service chịu trách nhiệm:

- Gắn access token vào request.
- Xử lý lỗi.
- Gọi refresh token khi cần.
- Điều hướng về Login nếu phiên hết hạn.
- Chuẩn hóa cách xử lý response.

Bên cạnh REST API, Frontend sử dụng WebSocket service để nhận các sự kiện realtime như violation mới, review update, delete violation, alert và telemetry.

3.7. Thiết kế dữ liệu

Hệ thống sử dụng kết hợp PostgreSQL, MongoDB và Cloudinary. Việc tách dữ liệu theo đặc điểm giúp hệ thống linh hoạt hơn.

3.7.1. Dữ liệu PostgreSQL

PostgreSQL lưu các dữ liệu có cấu trúc quan hệ rõ ràng.

3.7.1.1. User

Bảng User lưu thông tin tài khoản người dùng, bao gồm các thuộc tính:

- id
- username
- email
- hashed_password
- role
- is_active
- is_verified
- created_at
- updated_at
- reset_token
- reset_token_expires

User là trung tâm của các nghiệp vụ xác thực, phân quyền, review violation và audit log.

3.7.1.2. Refresh Token

Bảng Refresh Token lưu thông tin refresh token đã phát hành, bao gồm các thuộc tính:

- id
- user_id
- token_hash
- expires_at
- revoked_at
- created_at
- last_used_at
- user_agent
- ip_address

3.7.1.3. Camera

Bảng Camera lưu thông tin các nguồn camera, bao gồm các thuộc tính:

- id
- code
- name

- location
- source_type
- source_url
- is_active
- last_status
- last_checked_at
- is_deleted
- deleted_at

Camera sử dụng soft-delete để giữ ngữ cảnh cho các violation cũ.

3.7.1.4. Audit Log

Bảng Audit Log lưu lịch sử thao tác quan trọng, bao gồm các thuộc tính:

- id
- actor_id
- actor_username
- actor_role
- action
- target_type
- target_id
- description
- metadata_json
- ip_address
- created_at

Audit Log giúp hệ thống có khả năng truy vết và kiểm tra thao tác người dùng.

3.7.1.5. Security Alert

Bảng Alert lưu các cảnh báo bảo mật hoặc cảnh báo hệ thống, bao gồm các thuộc tính:

- id
- alert_type
- message
- severity

- created_at
- resolved_at

3.7.2. Dữ liệu MongoDB

MongoDB lưu các dữ liệu linh hoạt liên quan tới AI và thống kê.

3.7.2.1. Collection violations

Mỗi document violation lưu thông tin một lần phát hiện vi phạm, gồm các thuộc tính:

- _id
- timestamp
- image_url
- detections
- total_violations
- avg_conf
- status
- reviewed_by
- reviewed_at
- review_note
- rejection_reason
- camera_id
- camera_code
- camera_name
- camera_location
- camera_source_type
- is_demo

Trong đó, detections là mảng chứa các object gồm:

- class_id
- class_name
- confidence
- bbox
- track_id

3.7.2.2. Collection traffic_stats

Collection này lưu dữ liệu thống kê lưu lượng theo thời gian, bao gồm các thuộc tính:

- timestamp
- safe_count
- violation_count
- camera_id
- camera_code
- is_demo

Dữ liệu này được dùng để hiển thị thống kê tổng quan và biểu đồ xu hướng.

3.7.3. Dữ liệu Cloudinary

Cloudinary lưu các ảnh bằng chứng và ảnh đại diện.

MongoDB và PostgreSQL không lưu trực tiếp file ảnh, mà chỉ lưu URL hoặc metadata. Cách này giúp giảm tải cho server Backend và giúp Frontend hiển thị ảnh dễ dàng hơn.

Các nhóm ảnh chính gồm:

- Ảnh bằng chứng vi phạm.
- Ảnh đại diện người dùng.

3.8. Thiết kế các luồng xử lý chính

3.8.1. Luồng đăng nhập

1. Người dùng nhập username/password
2. Frontend gửi request login
3. Backend kiểm tra tài khoản và mật khẩu
4. Nếu hợp lệ, Backend tạo access token và refresh token
5. Refresh token được lưu vào cookie httpOnly
6. Access token được trả về Frontend
7. Frontend lưu access token và chuyển vào Dashboard

Luồng này giúp hệ thống xác thực người dùng trước khi cho phép truy cập các chức năng nghiệp vụ.

3.8.2. Luồng refresh token

Khi access token hết hạn, Frontend gọi API refresh token.

1. Access token hết hạn
2. Frontend gọi refresh API
3. Backend kiểm tra refresh token trong cookie
4. Nếu hợp lệ, revoke token cũ
5. Tạo refresh token mới
6. Tạo access token mới
7. Trả token mới cho Frontend

Cơ chế rotation giúp hạn chế việc refresh token cũ bị sử dụng lại.

3.8.3. Luồng xử lý ảnh upload

1. Người dùng upload ảnh
2. Frontend gửi file ảnh tới Backend
3. Backend kiểm tra định dạng file
4. Decode ảnh bằng OpenCV
5. YOLOv8s predict
6. Trích xuất detection
7. Vẽ bounding box lên ảnh
8. Nếu có Without Helmet, lưu violation
9. Trả kết quả cho Frontend

Luồng này phục vụ chức năng kiểm tra ảnh tĩnh.

3.8.4. Luồng livestream và phát hiện vi phạm

Đây là luồng xử lý quan trọng nhất của hệ thống.

1. Người dùng bật camera
2. Frontend mở MJPEG stream
3. Backend mở camera source
4. Capture frame bằng OpenCV
5. Chạy model tracking
6. Xác nhận vi phạm dựa trên nhiều frame
7. Upload ảnh evidence lên Cloundinary

8. Lưu violation vào MongoDB
9. Broadcast violation qua WebSocket
10. Frontend hiển thị thông báo realtime

Trong luồng này, hệ thống không ghi violation ngay khi một frame đơn lẻ dự đoán Without Helmet. Thay vào đó, hệ thống theo dõi đối tượng qua nhiều frame, kiểm tra tỷ lệ frame vi phạm và áp dụng các cơ chế chống trùng lặp trước khi lưu dữ liệu.

3.8.5. Luồng kiểm duyệt vi phạm

1. AI tạo violation ở trạng thái pending
2. Người dùng mở Violation History
3. Người dùng xem ảnh evidence và detection
4. Người dùng chọn Confirm hoặc Reject
5. Backend cập nhật status, reviewed_by, reviewed_at
6. Backend broadcast review_violation event
7. Frontend các tab liên quan cập nhật realtime

Ý nghĩa của luồng này là tách kết quả AI khỏi dữ liệu vi phạm chính thức. Chỉ các bản ghi confirmed mới được sử dụng trong báo cáo chính.

3.8.6. Luồng quản lý camera

- Admin tạo hoặc cập nhật camera
- Thông tin camera được lưu trong PostgreSQL
- Live Monitoring lấy danh sách camera sources
- Người dùng chọn camera
- Backend tra source_url từ database
- Pipeline video switch sang source mới
- Audit Log ghi nhận thao tác

Frontend không cần biết source_url thật của camera. Điều này giúp hạn chế rủi ro lộ thông tin RTSP hoặc đường dẫn nội bộ.

3.8.7. Luồng thống kê và báo cáo

1. Violation và traffic_stats được lưu trong MongoDB
2. Frontend gọi API report

3. Backend chạy aggregation pipeline
4. Chỉ tính violation đã confirmed
5. Trả dữ liệu tổng quan và biểu đồ
6. Frontend hiển thị Analytics

Đối với export Excel, Backend lấy dữ liệu phù hợp với bộ lọc, tạo file trong bộ nhớ và trả về cho người dùng tải xuống.

Đối với AI Feedback Dataset, Backend xuất dữ liệu đã được review thành file ZIP gồm manifest.csv, các file JSON detection và ảnh bằng chứng nếu người dùng chọn include images.

3.9. Thiết kế AI Pipeline trong livestream

AI Pipeline trong livestream được thiết kế để cân bằng giữa độ chính xác và hiệu năng realtime.

Các bước chính gồm:

1. Frame từ camera
2. YOLOv8s tracking
3. Lấy class, confidence, bbox, track_id
4. Cập nhật track_history
5. Kiểm tra confirmation logic
6. Kiểm tra cooldown/dedup
7. Lưu violation nếu hợp lệ

3.9.1. Track Voting

Track Voting là cơ chế xác nhận vi phạm theo lịch sử nhiều frame.

Thay vì tin vào một dự đoán duy nhất, hệ thống lưu lịch sử detection của từng track_id. Một đối tượng chỉ được xem là vi phạm khi:

- Xuất hiện đủ số frame tối thiểu.
- Tỷ lệ frame dự đoán Without Helmet vượt ngưỡng cấu hình.
- Confidence đủ tin cậy.

Cơ chế này giúp giảm các lỗi nhận diện tức thời.

3.9.2. Cooldown và Dedup

Sau khi một violation được xác nhận, hệ thống vẫn cần kiểm tra xem bản ghi này có bị trùng với bản ghi trước đó hay không.

Các điều kiện kiểm tra gồm:

- Khoảng thời gian giữa hai violation.
- Mức độ trùng khớp bounding box thông qua IoU.
- Khoảng cách giữa tâm bounding box.
- Camera source.

Nếu detection mới quá gần detection cũ, hệ thống bỏ qua để tránh ghi trùng.

3.9.3. Traffic Statistics

Trong khi xử lý livestream, hệ thống đồng thời thống kê lưu lượng.

Các đối tượng rời khỏi khung hình mà không bị đánh dấu vi phạm sẽ được tính là safe. Các vi phạm hợp lệ sau khi qua cooldown/dedup sẽ được tính vào violation count.

Dữ liệu được gom vào buffer và flush theo chu kỳ để tránh ghi MongoDB quá thường xuyên.

3.10. Thiết kế bảo mật hệ thống

3.10.1. Bảo vệ API bằng JWT

Các API nghiệp vụ yêu cầu access token hợp lệ. Backend kiểm tra token trước khi cho phép truy cập tài nguyên.

Access token có thời hạn ngắn để giảm rủi ro nếu bị lộ.

3.10.2. Refresh Token Rotation

Refresh token được lưu trong cookie httpOnly và database chỉ lưu token hash. Khi refresh, token cũ bị revoke và token mới được tạo ra.

Cơ chế này giúp tăng mức độ an toàn cho phiên đăng nhập.

3.10.3. Phân quyền theo vai trò

Hệ thống phân quyền theo hai vai trò chính:

- Admin
- Guard

Các API nhạy cảm như quản lý người dùng, quản lý camera, audit log chỉ cho phép Admin truy cập.

3.10.4. Bảo vệ thông tin camera

RTSP URL có thể chứa thông tin nhạy cảm. Vì vậy, Frontend chỉ nhận metadata như code, name, location và status. Source URL thật chỉ được Backend sử dụng nội bộ khi mở camera.

3.10.5. Audit Log

Các thao tác quan trọng được ghi Audit Log. Điều này giúp truy vết khi có thay đổi dữ liệu hoặc hành động bất thường.

3.11. Thiết kế triển khai và cấu hình

Hệ thống sử dụng file môi trường .env để quản lý cấu hình.

Các nhóm cấu hình chính gồm:

- Cấu hình PostgreSQL.
- Cấu hình MongoDB.
- Cấu hình Cloudinary.
- Cấu hình JWT.
- Cấu hình SMTP Email.
- Cấu hình CORS.
- Cấu hình model inference.
- Cấu hình camera và demo video.

PostgreSQL schema được quản lý bằng Alembic. Khi triển khai trên môi trường mới, cần chạy migration trước khi khởi động ứng dụng.

Việc tách migration khỏi runtime giúp hệ thống rõ ràng hơn và phù hợp hơn với môi trường production.

3.12. Nhận xét về thiết kế hệ thống

Thiết kế hệ thống trong đề tài hướng tới việc cân bằng giữa khả năng nhận diện AI, tính realtime và khả năng quản trị dữ liệu.

Một số điểm nổi bật của thiết kế gồm:

- Tách riêng Frontend, Backend, AI pipeline và lưu trữ dữ liệu.
- Kết hợp PostgreSQL và MongoDB để phù hợp với từng loại dữ liệu.
- Dùng Cloudinary để lưu ảnh bằng chứng thay vì lưu trực tiếp trên server.
- Dùng WebSocket để cập nhật sự kiện realtime.
- Dùng Review Workflow để tăng độ tin cậy của dữ liệu báo cáo.
- Dùng Camera Management để quản lý nhiều nguồn camera.
- Dùng Audit Log để truy vết thao tác người dùng.
- Dùng Demo Mode để hỗ trợ trình diễn hệ thống ổn định.
- Dùng Alembic để quản lý schema PostgreSQL rõ ràng hơn.

Nhìn chung, hệ thống được thiết kế không chỉ để chứng minh khả năng nhận diện bằng mô hình YOLOv8, mà còn hướng tới một ứng dụng web AI hoàn chỉnh có khả năng vận hành, quản trị và mở rộng trong thực tế.

4. Kết quả cài đặt và thử nghiệm

4.1. Môi trường cài đặt và triển khai thử nghiệm

Hệ thống được cài đặt và thử nghiệm trên máy cá nhân với cấu hình phần cứng có hỗ trợ GPU, phục vụ cho quá trình huấn luyện mô hình, kiểm thử AI inference và chạy thử hệ thống web.

Môi trường phát triển chính bao gồm:

- Hệ điều hành: Windows.
- Ngôn ngữ lập trình: Python, JavaScript.
- Backend: FastAPI.
- Frontend: React/Vite.
- Mô hình AI: YOLOv8.
- Cơ sở dữ liệu quan hệ: PostgreSQL.
- Cơ sở dữ liệu NoSQL: MongoDB.
- Lưu trữ ảnh: Cloudinary.
- Công cụ quản lý migration: Alembic.
- Camera/video: Webcam, RTSP camera và video demo.
- GPU: NVIDIA RTX 3050 Ti Laptop GPU.

Việc thử nghiệm được thực hiện theo nhiều giai đoạn, từ huấn luyện mô hình AI, xây dựng API Backend, phát triển giao diện Frontend, tích hợp camera realtime, kiểm thử

dữ liệu vi phạm, kiểm thử phân quyền, kiểm thử Analytics và kiểm thử các chức năng quản trị như Camera Management, Audit Log và Review Workflow.

4.2. Kết quả huấn luyện và đánh giá mô hình AI

4.2.1. Kết quả huấn luyện YOLOv8n

Ban đầu, em sử dụng mô hình YOLOv8n để làm mô hình baseline. Đây là phiên bản nhỏ nhất trong nhóm mô hình YOLOv8, có ưu điểm là tốc độ nhanh và yêu cầu tài nguyên thấp.

Quá trình huấn luyện YOLOv8n được thực hiện theo nhiều bước:

- Sanity check 2 epochs để kiểm tra dataset và pipeline.
- Debug training 20 epochs để kiểm tra khả năng học của mô hình.
- Final training 100 epochs để thu được trọng số tốt hơn.

Kết quả validation của mô hình YOLOv8n sau khi huấn luyện 100 epochs đạt:

Chỉ số	Giá trị
Precision	0.776
Recall	0.805
mAP@0.5	0.826
mAP@0.5:0.95	0.389

Kết quả này cho thấy mô hình YOLOv8n đã học được đặc trưng của bài toán và có khả năng phát hiện hai lớp With Helmet và Without Helmet. Tuy nhiên, mô hình vẫn còn hạn chế ở lớp Without Helmet, đặc biệt là khả năng bỏ sót một số trường hợp vi phạm trong điều kiện ảnh khó.

4.2.2. Kết quả huấn luyện YOLOv8s 100 epochs

Sau khi đánh giá mô hình YOLOv8n, em tiếp tục huấn luyện mô hình YOLOv8s nhằm cải thiện độ chính xác. YOLOv8s có kích thước lớn hơn YOLOv8n, số lượng tham số nhiều hơn và khả năng học đặc trưng tốt hơn.

Kết quả validation của mô hình YOLOv8s 100 epochs:

Chỉ số	Giá trị
Precision	0.796

Recall	0.837
mAP@0.5	0.862
mAP@0.5:0.95	0.442

Kết quả theo từng lớp:

Lớp	Precision	Recall	mAP@0.5	mAP@0.5:0.95
With Helmet	0.84	0.88	0.902	0.486
Without Helmet	0.752	0.793	0.821	0.397

So với YOLOv8n, mô hình YOLOv8s cho kết quả tốt hơn ở hầu hết các chỉ số. Đặc biệt, lớp Without Helmet được cải thiện rõ rệt, giúp hệ thống giảm khả năng bỏ sót vi phạm. Đây là cơ sở để lựa chọn YOLOv8s làm mô hình chính cho hệ thống thay vì YOLOv8n.

4.2.3. Kết quả huấn luyện YOLOv8s 200 epochs

Để tiếp tục kiểm tra giới hạn hội tụ của mô hình YOLOv8s, em tiến hành huấn luyện thêm với số lượng epoch lớn hơn. Quá trình huấn luyện sử dụng cơ chế Early Stopping để dừng sớm nếu mô hình không còn cải thiện.

Trong quá trình huấn luyện, mô hình dừng tại epoch 128 do không còn cải thiện đáng kể sau nhiều epoch liên tiếp. Điều này cho thấy cơ chế Early Stopping đã hoạt động đúng, giúp tránh huấn luyện dư thừa và hạn chế nguy cơ overfitting.

Bảng so sánh kết quả YOLOv8s 200 epochs và YOLOv8s 100 epochs trên tập validation:

Chỉ số	200	100	Độ chênh lệch
Precision	0.864	0.796	+0.068
Recall	0.842	0.837	+0.005
mAP@0.5	0.875	0.862	+0.013
mAP@0.5:0.95	0.469	0.442	+0.027

Kết quả cho thấy phiên bản huấn luyện mở rộng đạt cải thiện đáng kể về Precision và mAP@0.5. Precision tăng từ 0.796 lên 0.864, chứng tỏ mô hình giảm được nhiều dự đoán sai. Chỉ số mAP@0.5 đạt 0.875, gần đạt mốc 90%, thể hiện mô hình đã hội tụ tốt hơn so với bản 100 epochs.

Từ kết quả trên, trọng số best.pt của mô hình YOLOv8s 200 epochs được lựa chọn làm mô hình nhận diện chính của hệ thống.

4.2.4. Nhận xét về kết quả mô hình

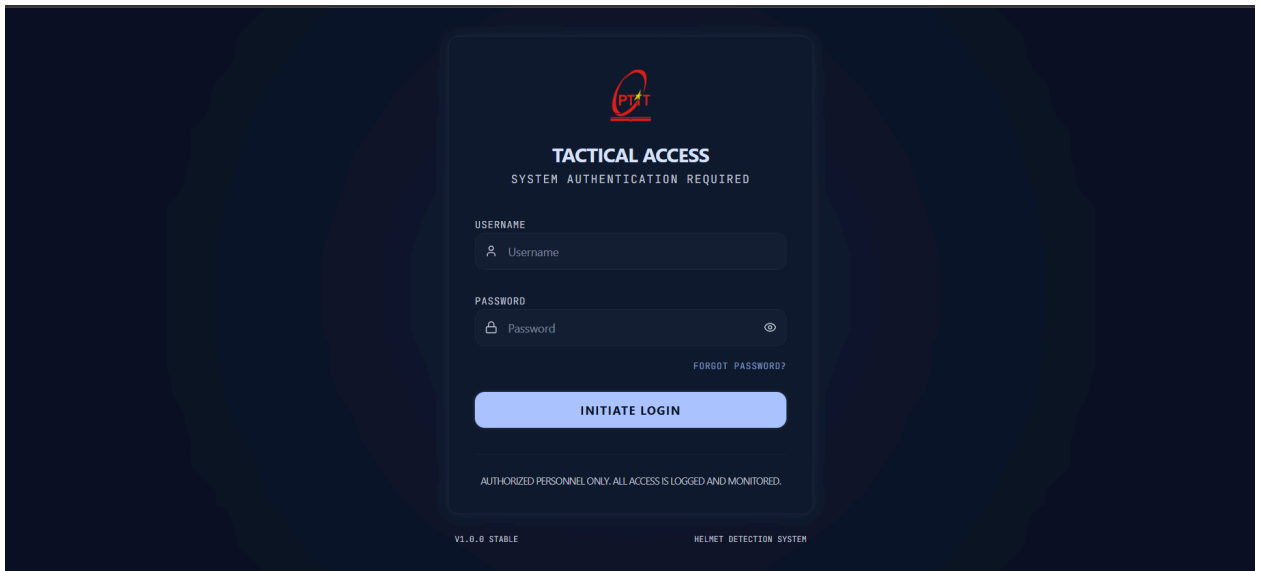
Qua quá trình thử nghiệm, có thể rút ra một số nhận xét:

- YOLOv8n có tốc độ nhanh nhưng độ chính xác chưa cao bằng YOLOv8s.
- YOLOv8s cải thiện tốt hơn ở lớp Without Helmet, là lớp quan trọng nhất của đề tài.
- Việc tăng số epoch kết hợp Early Stopping giúp mô hình YOLOv8s đạt kết quả tốt hơn mà không cần huấn luyện đủ toàn bộ 200 epochs.
- Chỉ số Precision tăng mạnh giúp giảm cảnh báo sai.
- mAP@0.5 gần đạt 90% cho thấy mô hình đủ tốt để tích hợp vào hệ thống web giám sát realtime.

Tuy nhiên, trong điều kiện thực tế như webcam mờ, thiếu sáng, góc quay cao hoặc đối tượng bị che khuất, mô hình vẫn có thể sinh ra một số kết quả chưa chính xác. Vì vậy, hệ thống không sử dụng kết quả AI như vi phạm chính thức ngay lập tức, mà bổ sung thêm Review Workflow để người dùng kiểm duyệt lại dữ liệu.

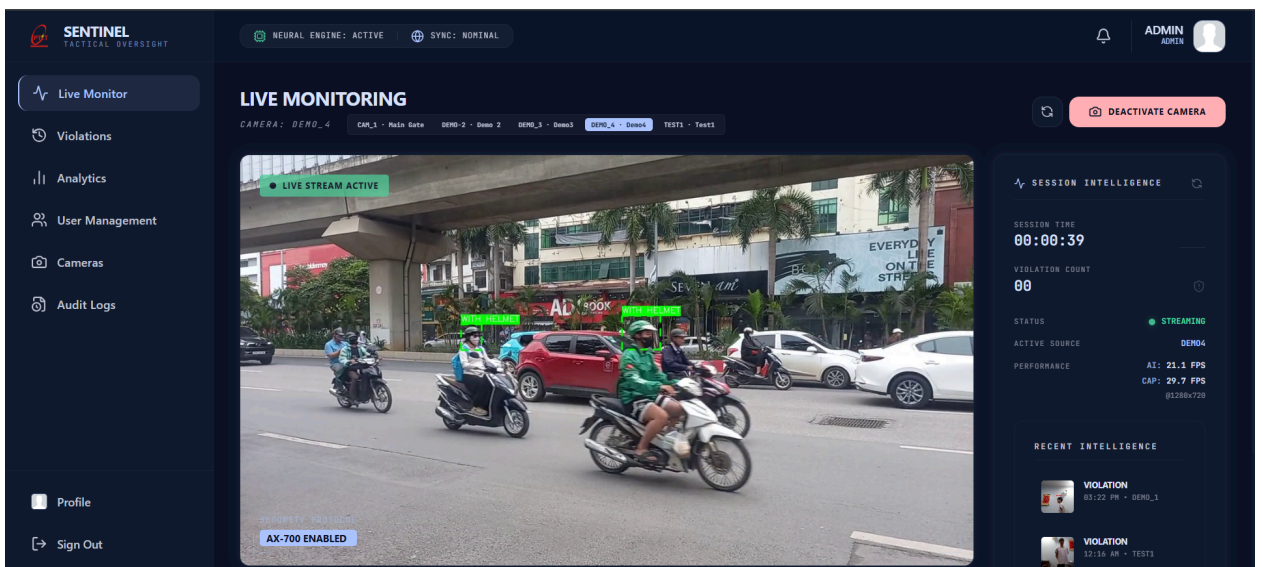
4.3. Kết quả cài đặt hệ thống web

Sau quá trình phát triển, hệ thống đã được hoàn thiện thành một ứng dụng web fullstack có khả năng nhận diện người không đội mũ bảo hiểm, hiển thị livestream, lưu ảnh bằng chứng, quản lý lịch sử vi phạm, kiểm duyệt kết quả AI, thống kê dữ liệu và quản trị hệ thống.



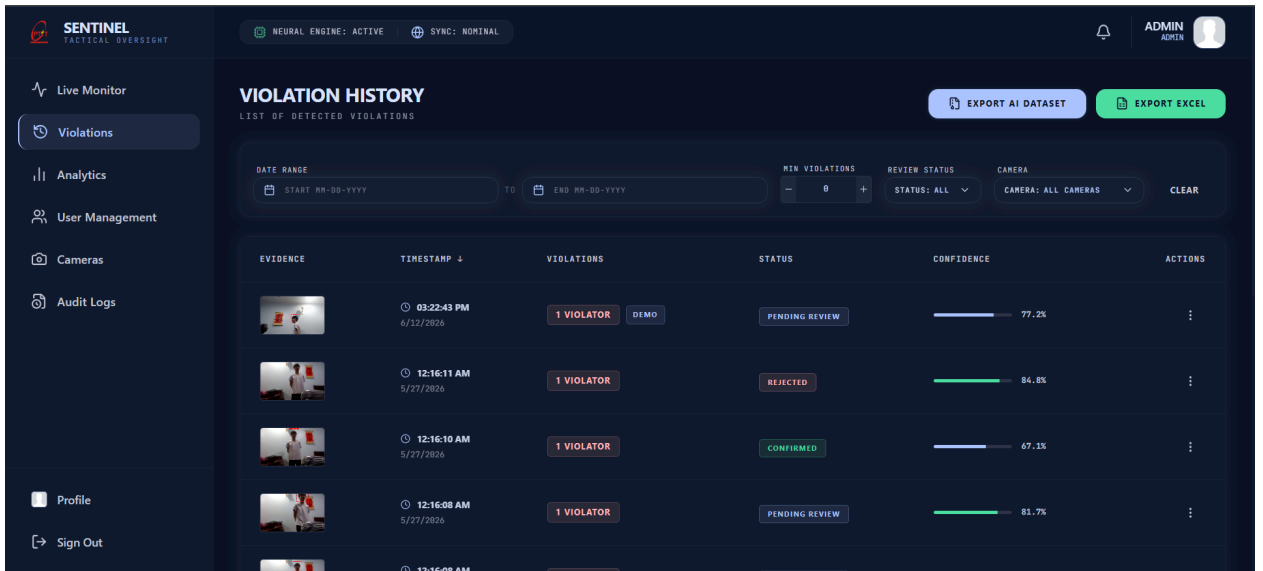
Hình 4.1. Giao diện đăng nhập hệ thống

Mô tả: giao diện cho phép người dùng đăng nhập và truy cập hệ thống theo quyền được cấp.



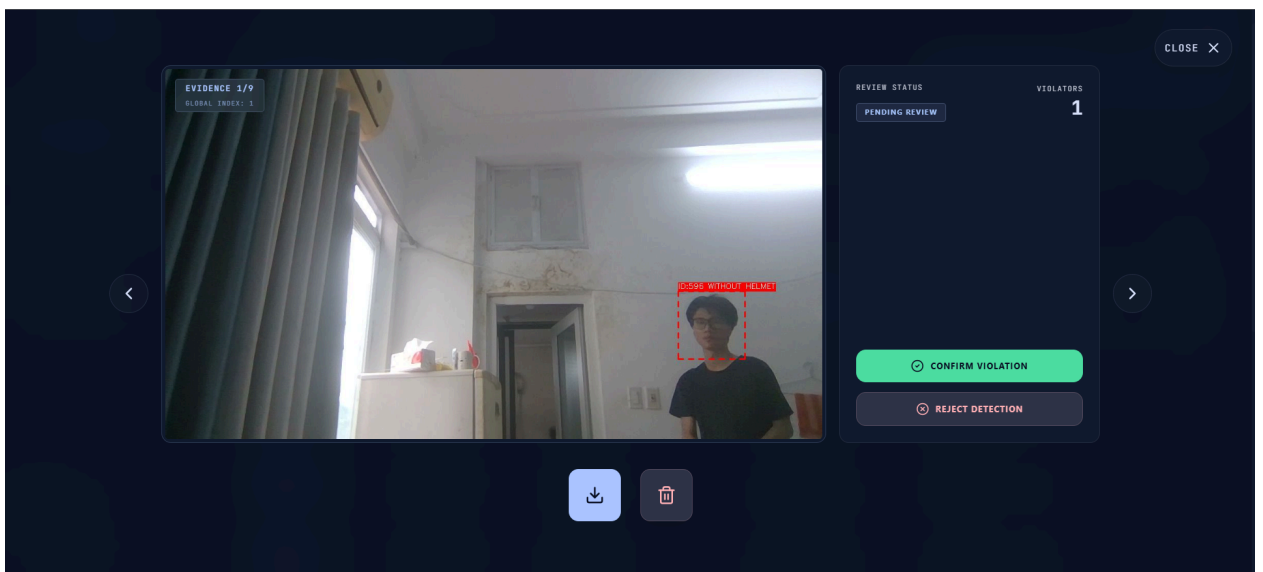
Hình 4.2. Giao diện Live Monitoring

Mô tả: hiển thị livestream camera, bounding box, trạng thái camera, AI FPS, Capture FPS và thông báo vi phạm realtime.



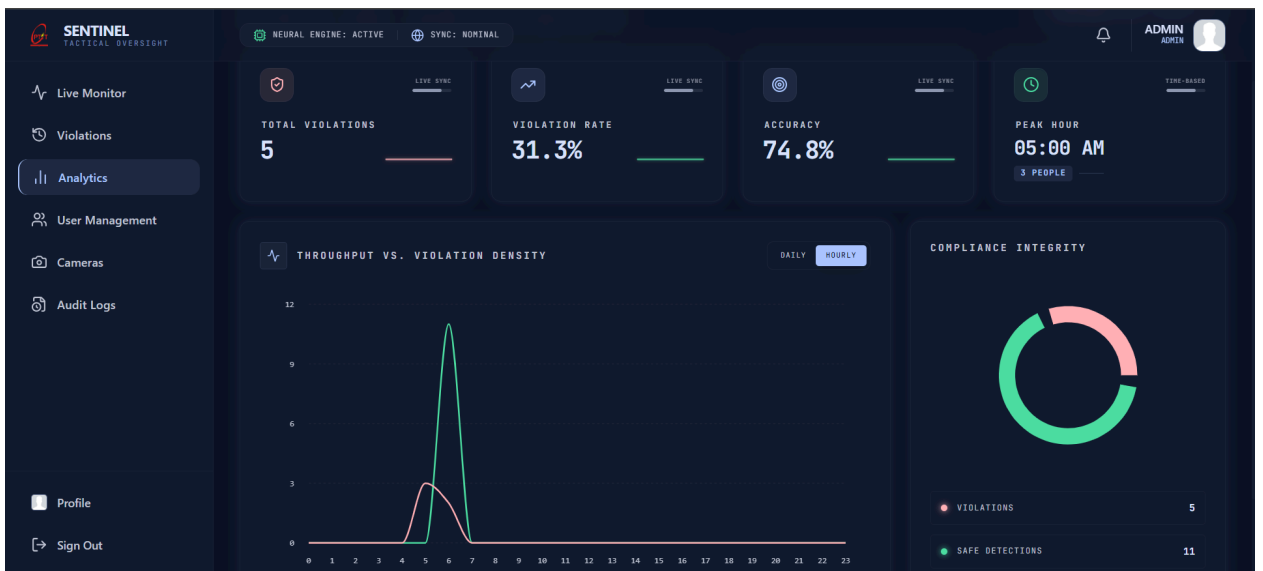
Hình 4.3. Giao diện Violation History

Mô tả: hiển thị danh sách vi phạm, ảnh bằng chứng, trạng thái review và các bộ lọc dữ liệu.



Hình 4.4. Giao diện Review Violation

Mô tả: người dùng có thể xác nhận hoặc từ chối vi phạm do AI phát hiện.



Hình 4.5. Giao diện Analytics

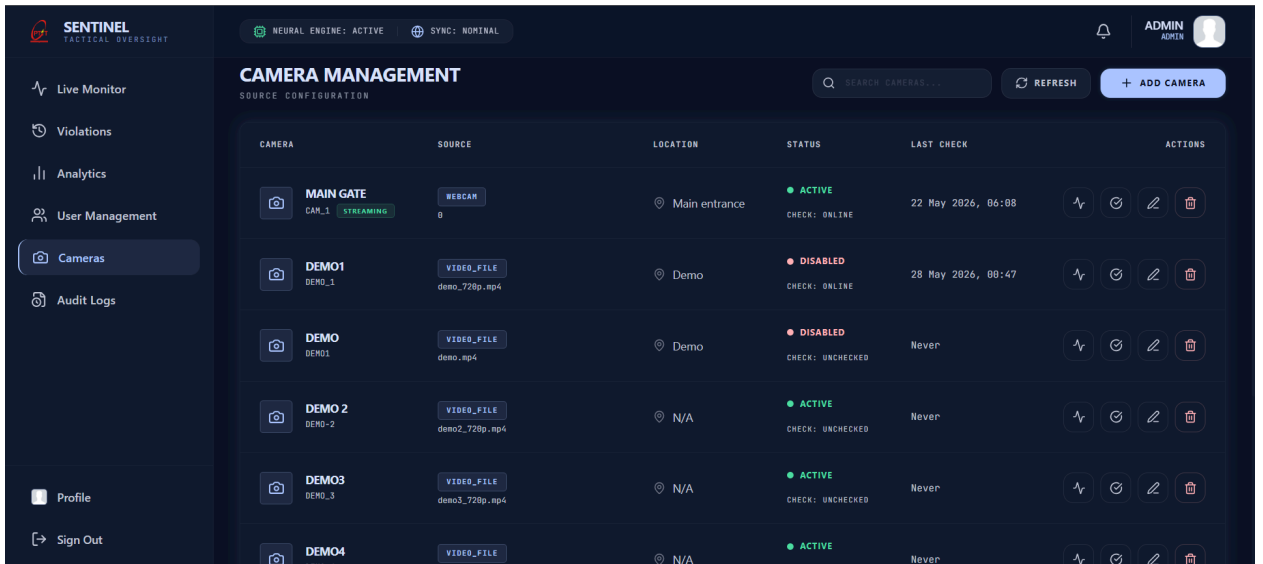
Mô tả: hiển thị thống kê số vi phạm, số người an toàn, độ chính xác trung bình và biểu đồ xu hướng.

The User Management interface allows administrators to manage system users. The table below shows the current user list:

USER	CREDENTIALS	ROLE	VERIFIED	STATUS	ACTIONS
ADMIN ID: 1	@admin admin@gmail.com	ADMIN	VERIFIED	ACTIVE	⋮
GUARD1 ID: 2	@guard1 guard1@gmail.com	GUARD	VERIFIED	ACTIVE	⋮
GUARD2 ID: 3	@guard2 guard2@gmail.com	GUARD	UNVERIFIED	ACTIVE	⋮
ADMIN2 ID: 4	@admin2 admin2@gmail.com	ADMIN	VERIFIED	ACTIVE	⋮
GUARD3 ID: 5	@guard3 guard3@gmail.com	GUARD	UNVERIFIED	INACTIVE	⋮

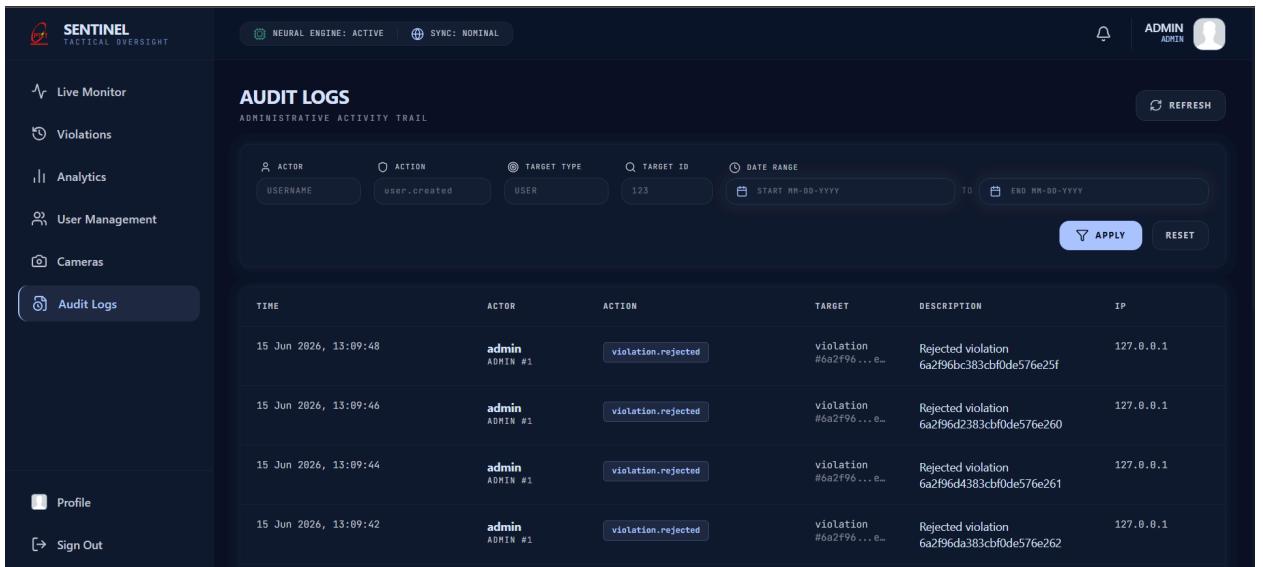
Hình 4.6. Giao diện User Management

Mô tả: Admin có thể thêm, thay đổi trạng thái hoạt động của các tài khoản



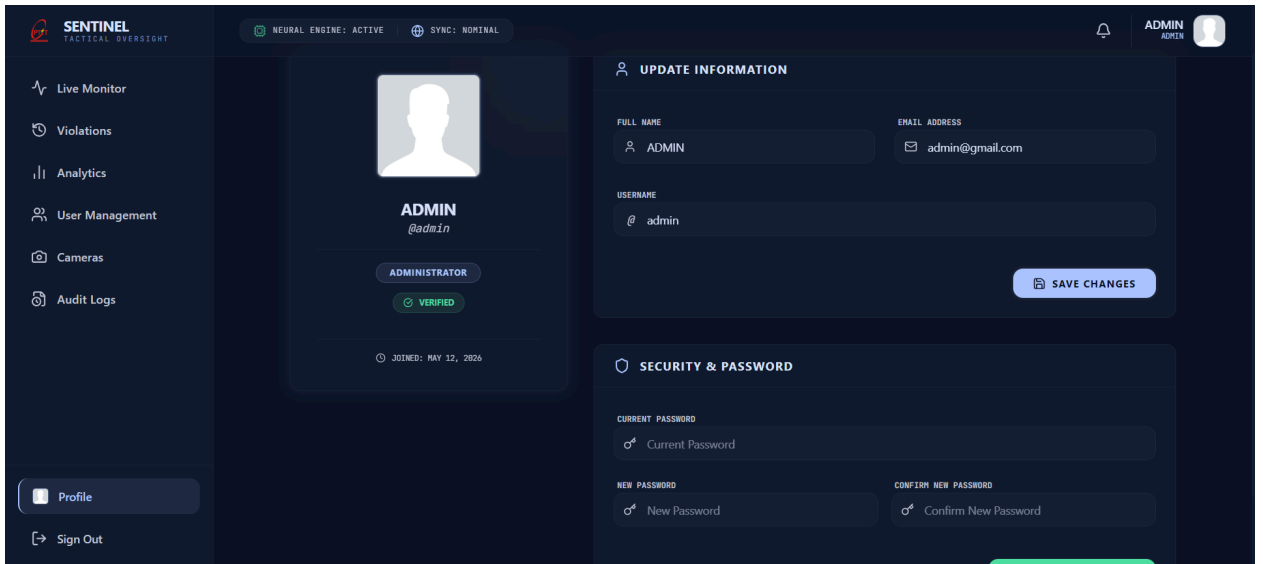
Hình 4.7. Giao diện Camera Management

Mô tả: Admin có thể thêm, sửa, bật/tắt, test connection và quản lý các nguồn camera.



Hình 4.8. Giao diện Audit Logs

Mô tả: ghi lại các thao tác quan trọng của người dùng trong hệ thống.



Hình 4.9. Giao diện trang cá nhân

Mô tả: người dùng có thể thay đổi thông tin, thay đổi ảnh đại diện hoặc thay đổi mật khẩu tài khoản của mình

5. Kết luận

5.1. Nhận xét chung về hệ thống

Sau quá trình thực hiện đề tài, hệ thống web phát hiện người không đội mũ bảo hiểm bằng mô hình YOLOv8 đã được xây dựng tương đối hoàn chỉnh, đáp ứng được các mục tiêu chính đặt ra ban đầu. Hệ thống không chỉ dừng lại ở việc huấn luyện mô hình AI để nhận diện người đội mũ và không đội mũ bảo hiểm, mà còn được phát triển thành một ứng dụng web có đầy đủ các thành phần cần thiết như Backend, Frontend, Database, lưu trữ ảnh bằng chứng, xác thực người dùng, phân quyền, giám sát realtime, thống kê báo cáo và quản trị hệ thống.

Về mặt mô hình AI, đề tài đã thực hiện quá trình tìm hiểu, chuẩn bị dataset, huấn luyện và đánh giá nhiều phiên bản YOLOv8 khác nhau. Ban đầu, mô hình YOLOv8n được sử dụng làm baseline vì có tốc độ nhanh và yêu cầu tài nguyên thấp. Sau đó, mô hình YOLOv8s được huấn luyện và đánh giá để cải thiện độ chính xác. Kết quả thực nghiệm cho thấy YOLOv8s cho hiệu quả tốt hơn, đặc biệt ở lớp “Without Helmet”, là lớp quan trọng nhất của bài toán. Việc tiếp tục huấn luyện YOLOv8s với số epoch lớn hơn và sử dụng Early Stopping giúp mô hình đạt kết quả tốt hơn, với mAP@0.5 đạt 0.885 trên tập test.

Về mặt hệ thống, Backend được xây dựng bằng FastAPI, có khả năng xử lý ảnh upload, livestream camera, chạy AI inference, lưu dữ liệu vi phạm, gửi sự kiện realtime, quản lý người dùng, quản lý camera và xuất báo cáo. Frontend được xây dựng bằng React/Vite, cung cấp giao diện trực quan cho các chức năng như Live Monitoring, Violation History, Analytics, User Management, Camera Management và Audit Logs. Hệ thống sử dụng PostgreSQL cho dữ liệu quan hệ, MongoDB cho dữ liệu AI và Cloudinary để lưu trữ ảnh bằng chứng.

Một điểm quan trọng của hệ thống là không xem kết quả AI là dữ liệu vi phạm chính thức ngay lập tức. Thay vào đó, hệ thống áp dụng Review Workflow, trong đó các vi phạm do AI phát hiện ban đầu được lưu ở trạng thái pending, sau đó người dùng có thể xác nhận hoặc từ chối. Cách tiếp cận này giúp tăng độ tin cậy của dữ liệu báo cáo và phù hợp hơn với thực tế, vì mô hình AI vẫn có thể gặp sai số trong một số điều kiện khó.

Nhìn chung, hệ thống đã thể hiện được một quy trình xử lý tương đối đầy đủ từ camera hoặc ảnh đầu vào, qua mô hình AI, lưu ảnh bằng chứng, lưu metadata, hiển thị realtime, kiểm duyệt dữ liệu, thống kê và xuất báo cáo. Điều này cho thấy đề tài không chỉ mang tính thử nghiệm mô hình, mà còn có định hướng xây dựng một ứng dụng AI Web thực tế.

5.2. Kết quả đạt được

Trong quá trình thực hiện đề tài, em đã đạt được các kết quả chính sau:

- Tìm hiểu được các kiến thức nền tảng về Computer Vision, Object Detection, Deep Learning, CNN, YOLOv8 và các chỉ số đánh giá mô hình.
- Chuẩn bị được môi trường huấn luyện mô hình trên máy cá nhân có hỗ trợ GPU.
- Tìm kiếm, phân tích và sử dụng được dataset phát hiện mũ bảo hiểm theo định dạng YOLO.
- Huấn luyện và đánh giá được các mô hình YOLOv8n và YOLOv8s.
- So sánh được hiệu quả giữa các phiên bản mô hình và lựa chọn YOLOv8s làm mô hình chính của hệ thống.
- Xây dựng được Backend bằng FastAPI để xử lý AI inference, livestream, dữ liệu vi phạm, xác thực, phân quyền và báo cáo.

- Xây dựng được Frontend bằng React/Vite với các giao diện chính phục vụ giám sát và quản trị.
- Tích hợp được livestream camera, WebSocket realtime và Demo Mode.
- Lưu trữ được ảnh bằng chứng trên Cloudinary và metadata vi phạm trên MongoDB.
- Xây dựng được Review Workflow để người dùng kiểm duyệt kết quả AI.
- Xây dựng được Analytics, Export Excel và AI Feedback Dataset.
- Hoàn thiện được các chức năng quản trị như User Management, Camera Management và Audit Log.
- Chuyển hệ thống dữ liệu quan hệ từ SQLite sang PostgreSQL và quản lý schema bằng Alembic.

Các kết quả này cho thấy hệ thống đã hoàn thành được phần lớn yêu cầu đặt ra trong phạm vi môn Thực tập cơ sở. Sản phẩm cuối cùng có thể chạy thử nghiệm, demo trực tiếp và thể hiện được đầy đủ luồng xử lý chính của một hệ thống giám sát thông minh.

5.3. Hạn chế của hệ thống

Bên cạnh các kết quả đã đạt được, hệ thống vẫn còn một số hạn chế nhất định.

Thứ nhất, độ chính xác của mô hình AI vẫn phụ thuộc vào chất lượng ảnh đầu vào. Trong các điều kiện như thiếu sáng, ảnh bị mờ, góc quay cao, đối tượng bị che khuất hoặc camera có độ phân giải thấp, mô hình vẫn có thể nhận diện sai hoặc bỏ sót một số trường hợp vi phạm.

Thứ hai, dataset sử dụng trong quá trình huấn luyện tuy đã phù hợp với bài toán phát hiện mũ bảo hiểm, nhưng vẫn chưa hoàn toàn phản ánh đầy đủ tất cả các điều kiện thực tế tại Việt Nam. Các tình huống như đường đông, nhiều người chồng lấp, mũ có hình dạng đặc biệt, ánh sáng ban đêm hoặc góc quay từ camera giám sát thực tế vẫn cần được bổ sung thêm để cải thiện khả năng tổng quát hóa của mô hình.

Thứ ba, hệ thống chủ yếu được thử nghiệm trong môi trường local, chưa được đánh giá đầy đủ trong môi trường production với nhiều camera, nhiều người dùng đồng thời và thời gian vận hành dài. Do đó, khả năng chịu tải và độ ổn định khi triển khai thực tế ở quy mô lớn vẫn cần được kiểm thử thêm.

Thứ tư, mặc dù hệ thống đã có Camera Management và Demo Mode, việc quản lý camera ở quy mô lớn vẫn có thể cần bổ sung thêm các chức năng nâng cao như theo dõi trạng thái camera định kỳ, cảnh báo camera mất kết nối, phân nhóm camera theo khu vực hoặc phân quyền camera theo người dùng.

Cuối cùng, hệ thống hiện tại mới tập trung vào bài toán phát hiện người không đội mũ bảo hiểm. Các hành vi vi phạm giao thông khác như vượt đèn đỏ, đi sai làn, chờ quá số người quy định hoặc sử dụng điện thoại khi lái xe chưa được hỗ trợ.

5.4. Hướng phát triển trong tương lai

Từ các hạn chế trên, hệ thống có thể tiếp tục được phát triển theo một số hướng sau:

- Bổ sung thêm dữ liệu thực tế từ nhiều điều kiện camera khác nhau để fine-tune mô hình, đặc biệt là các trường hợp thiếu sáng, ảnh mờ, góc quay cao và nhiều đối tượng chồng lấp.
- Tận dụng AI Feedback Dataset từ các bản ghi đã được người dùng confirm/reject để cải thiện mô hình theo hướng học từ dữ liệu vận hành thực tế.
- Triển khai hệ thống trên môi trường production hoặc server riêng để đánh giá khả năng vận hành dài hạn.
- Tối ưu hiệu năng xử lý realtime khi có nhiều camera hoạt động đồng thời.
- Bổ sung cơ chế cảnh báo camera mất kết nối hoặc camera hoạt động bất thường.
- Mở rộng hệ thống để nhận diện thêm các hành vi vi phạm giao thông khác.
- Hoàn thiện hơn các chức năng báo cáo, ví dụ báo cáo theo khu vực camera, theo khung giờ cao điểm hoặc theo tỷ lệ vi phạm.
- Nghiên cứu triển khai các cơ chế realtime nâng cao hơn như WebRTC nếu cần giảm độ trễ video ở quy mô lớn.

Những hướng phát triển này có thể giúp hệ thống tiến gần hơn tới một sản phẩm giám sát giao thông thông minh có khả năng ứng dụng thực tế.

5.5. Kết luận

Thông qua đề tài “Xây dựng hệ thống web phát hiện người không đội mũ bảo hiểm bằng mô hình YOLOv8”, em đã có cơ hội vận dụng tổng hợp nhiều kiến thức liên quan đến trí tuệ nhân tạo, thị giác máy tính và phát triển phần mềm web. Quá trình thực hiện đề tài giúp em hiểu rõ hơn về quy trình xây dựng một hệ thống AI hoàn chỉnh, từ khâu chuẩn bị dữ liệu, huấn luyện mô hình, đánh giá kết quả, tích hợp mô hình vào Backend, xây dựng giao diện người dùng, lưu trữ dữ liệu, xử lý realtime cho đến quản lý và kiểm duyệt kết quả.

Kết quả cuối cùng của đề tài là một hệ thống web có khả năng nhận diện người không đội mũ bảo hiểm từ ảnh, video hoặc camera thời gian thực. Hệ thống có thể lưu ảnh bằng chứng, quản lý lịch sử vi phạm, hỗ trợ người dùng kiểm duyệt kết quả AI, thống kê dữ liệu, xuất báo cáo và quản trị các thành phần quan trọng như người dùng, camera và audit log.

Mặc dù hệ thống vẫn còn một số hạn chế và cần tiếp tục cải thiện để có thể triển khai thực tế ở quy mô lớn, đề tài đã hoàn thành được mục tiêu chính đề ra trong phạm vi môn Thực tập cơ sở. Đây là nền tảng quan trọng để em tiếp tục phát triển các hệ thống AI ứng dụng thực tế trong tương lai, đặc biệt là các hệ thống kết hợp giữa mô hình học sâu và ứng dụng web fullstack.

Tài liệu tham khảo

- [1] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [3] Ultralytics, “YOLOv8 Documentation,” Ultralytics Docs.
- [4] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” in Proc. European Conference on Computer Vision (ECCV), 2022.
- [5] OpenCV, “cv::VideoCapture Class Reference,” OpenCV Documentation.
- [6] FastAPI, “FastAPI Documentation,” FastAPI Official Documentation.
- [7] Meta Open Source, “React Documentation,” React Official Documentation.
- [8] PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL Official Website.
- [9] SQLAlchemy, “Alembic Documentation,” Alembic Official Documentation.
- [10] MongoDB, “Documents,” MongoDB Manual.
- [11] Cloudinary, “Upload API Reference,” Cloudinary Documentation.
- [12] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, 2015.

- [13] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, 2011.
- [14] J. Klensin, “Simple Mail Transfer Protocol,” RFC 5321, 2008.
- [15] OWASP Foundation, “Forgot Password Cheat Sheet,” OWASP Cheat Sheet Series.